CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

TRUY VẤN DỮ LIỆU TRÊN MICROSERVICE

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Microservices Data Management - Commands and Queries

Microservices Data Query Pattern and Best Practices

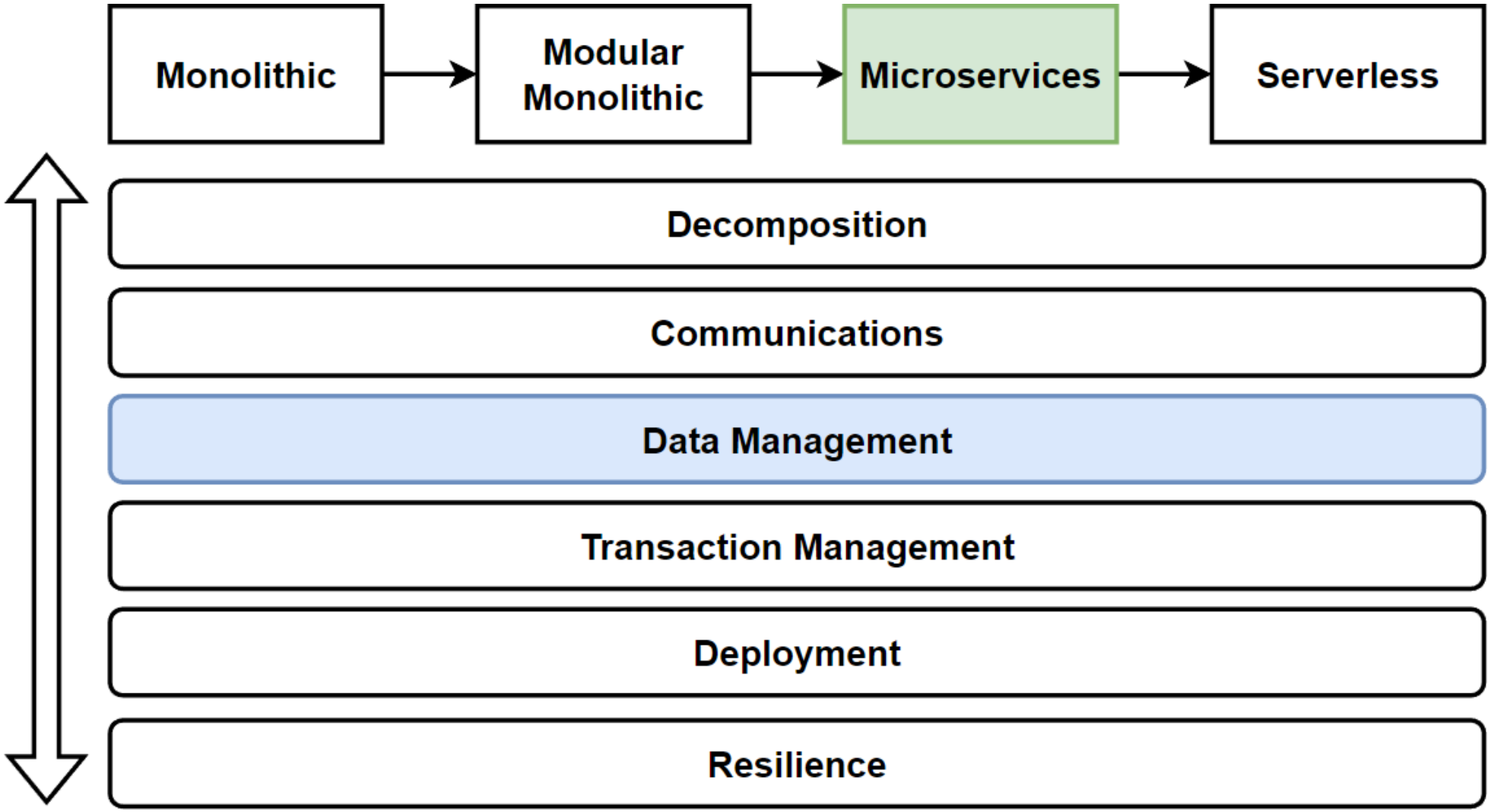
Materialized View Pattern

CQRS Design Pattern

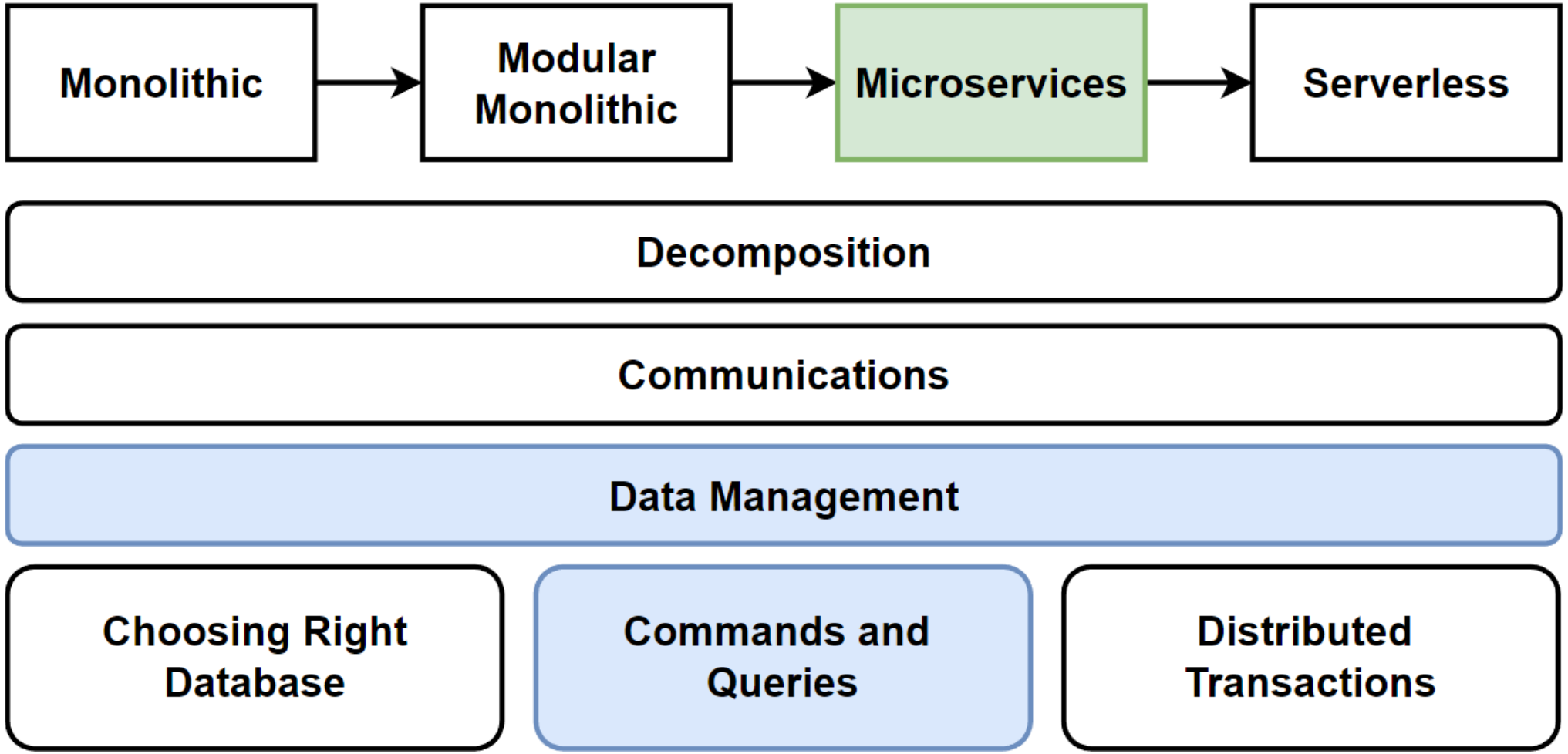
Event Sourcing Pattern

Eventual Consistency Principle

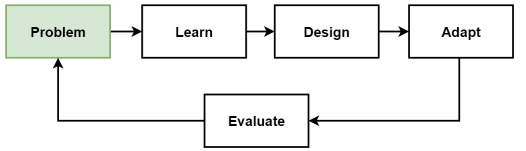
Architecture Design – Vertical Considerations



Microservices Data Management - Main Topics



Problem: Cross-Service Queries and Write Commands on Distributed Scaled Databases



Considerations

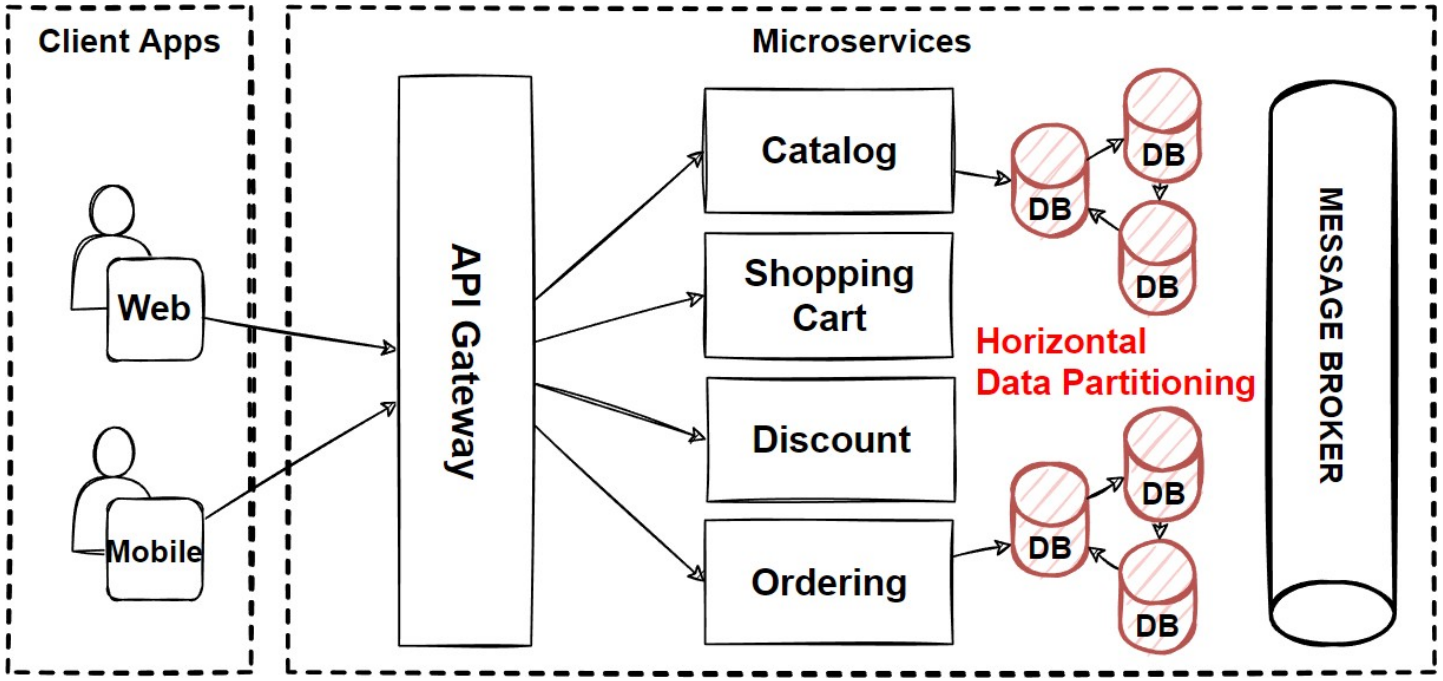
- Cross-services queries that retrieve data from several microservices ?
- Separate read and write operations at scale ?

Problems

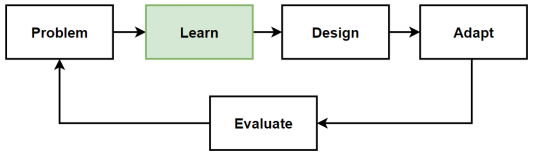
- Cross-Service Queries with Complex JOIN operations
- Read and write operations at scale
- Distributed Transaction Management

Solutions

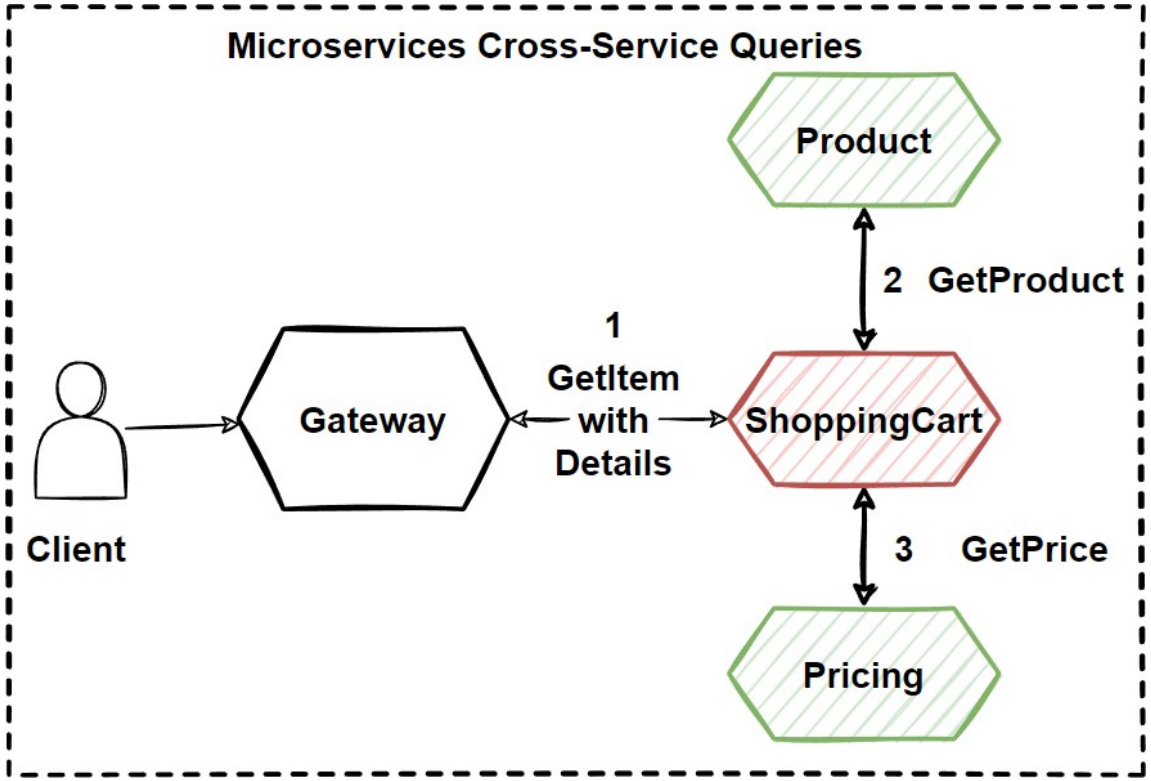
- Microservices Data Query Pattern and Best Practices
- Materialized View Pattern
- CQRS Design Pattern
- Event Sourcing Pattern



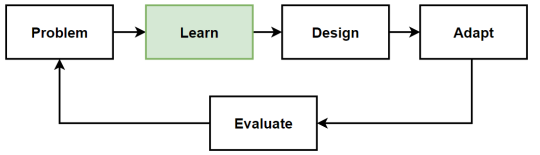
Microservices Cross-Service Queries



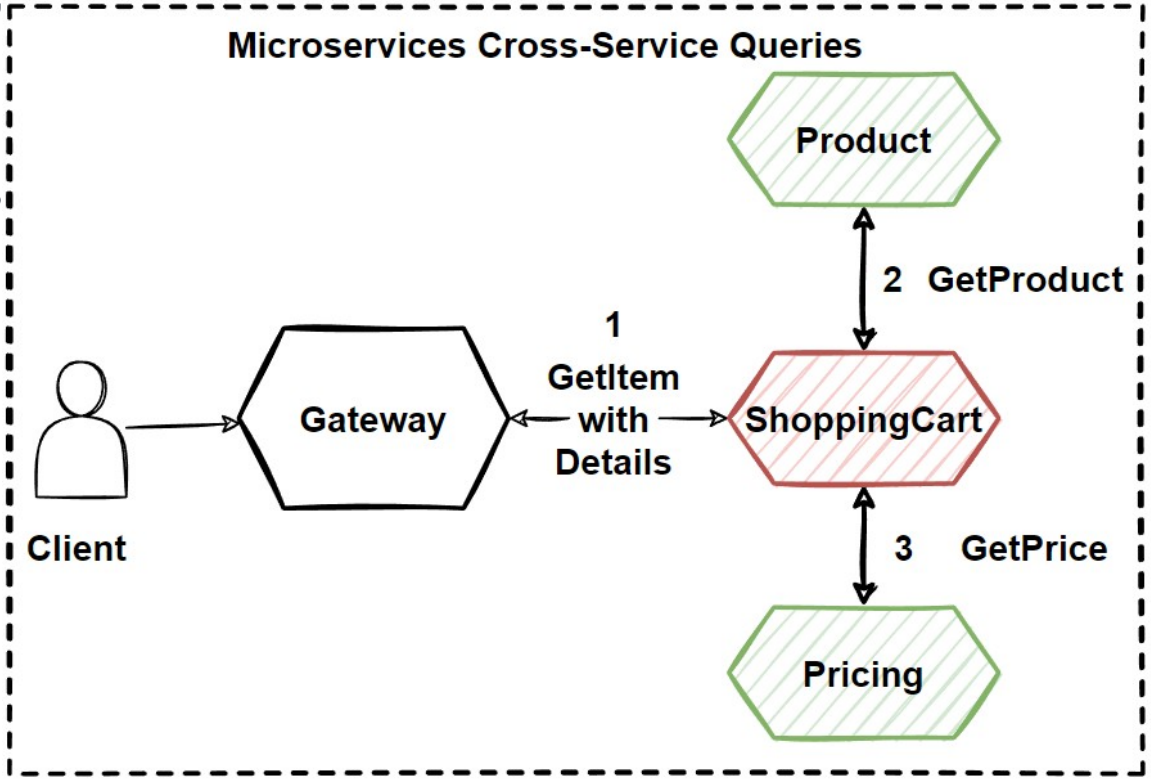
- **Monolithic architectures**, its very **easy to query** different entities, **Querying data** across multiple tables is **straightforward**.
- **Microservices architectures** uses **polyglot persistence**, has different databases, **need strategy** to manage queries.
- What if the client requests are **visit more than one** internal **microservices ?**
- **E-commerce application** we have **product**, **basket**, **discount**, **ordering** microservices that **needs to interact** each other to perform customer use cases.
- **Integrations** are **querying each services** data for aggregation or perform logics.



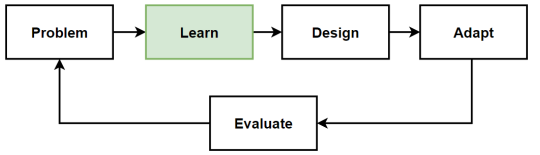
How can we manage these cross-services queries ?

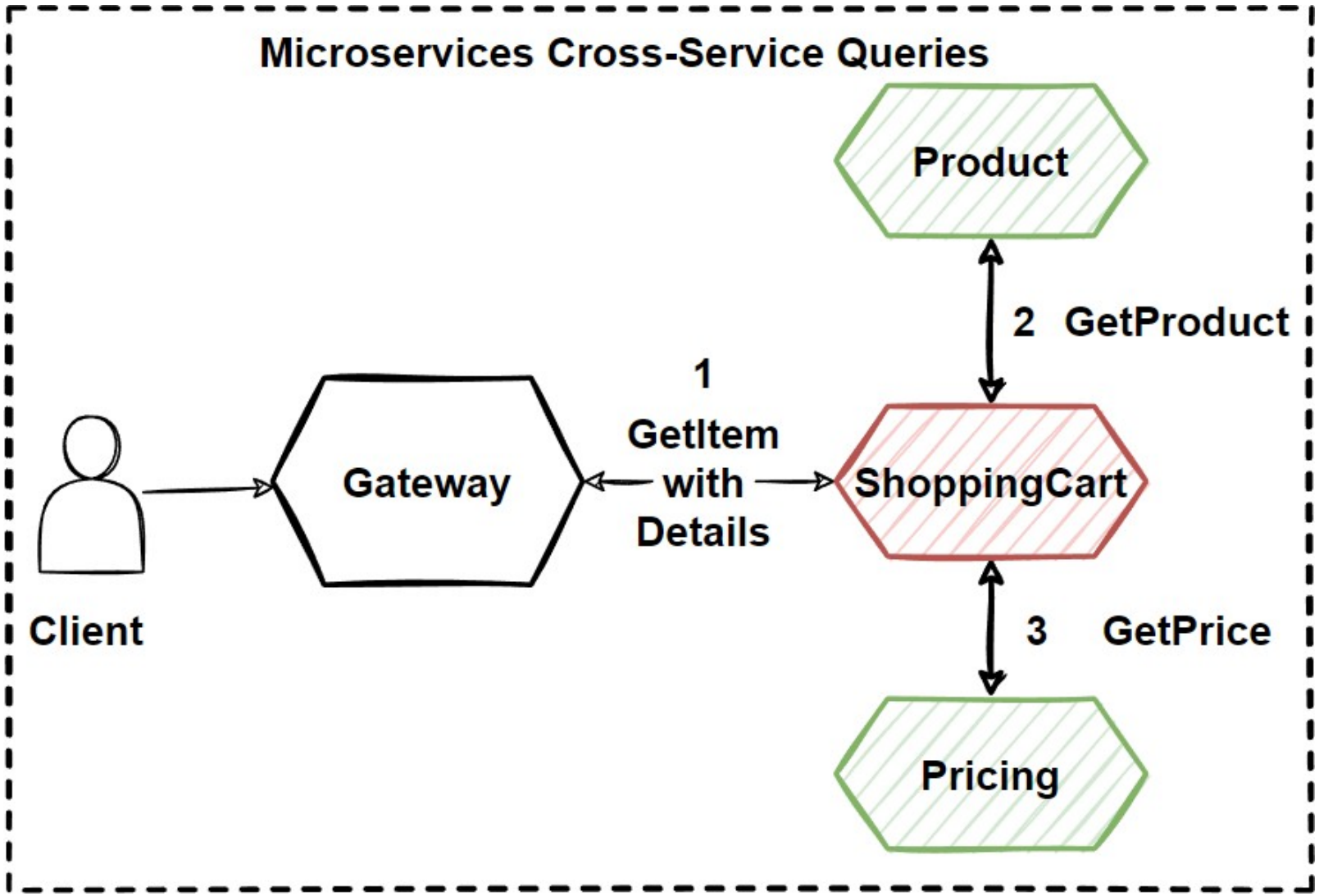


- **Direct HTTP Communication**
  Not a good solution that makes coupling each microservices, and loose power of microservices independency.
- **Async Communication**
  The best practice is reducing inter-service communication as much as possible and use async communication. Can't reduce these internal communications due to customer requirement.
- Client **send query** request to **internal microservices** to **accumulate** some **data**.
- Those query request wait **immediate response** so we can't proceed with async communication.
- **Transient errors**, **Network congestion** or any overloaded microservice can result in long-running and failed operations.
- **Materialized View Pattern**
  Reduce inter-service communication and provide sync response.

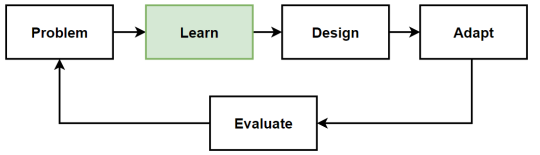


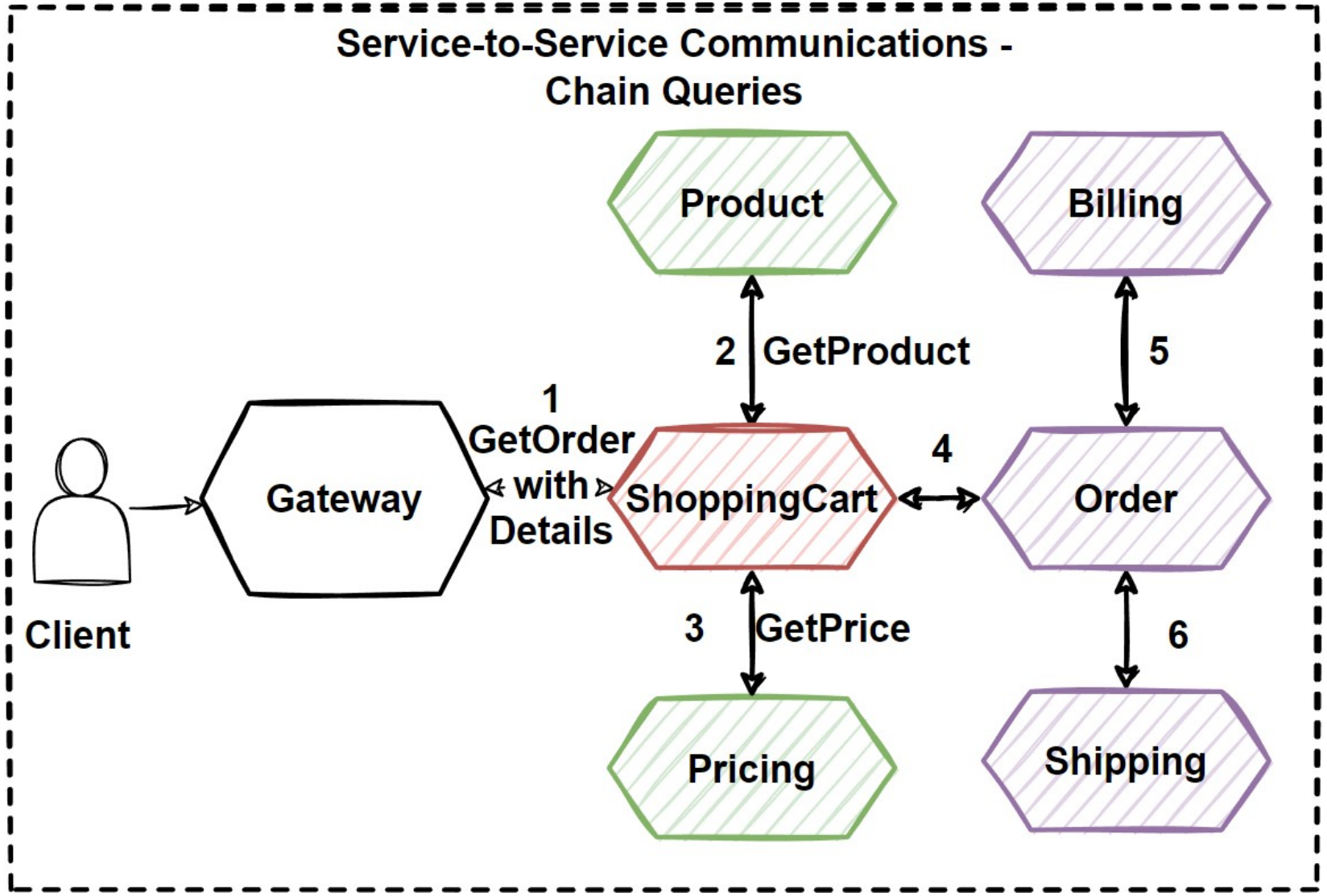
Microservices Cross-Service Queries
Example Use Case – Get Item with Details



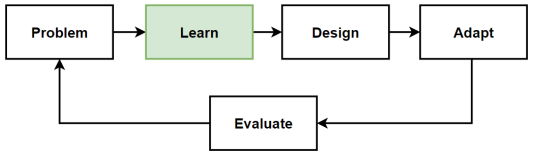


Microservices Cross-Service Queries
Chain Queries – Get Order with All Details

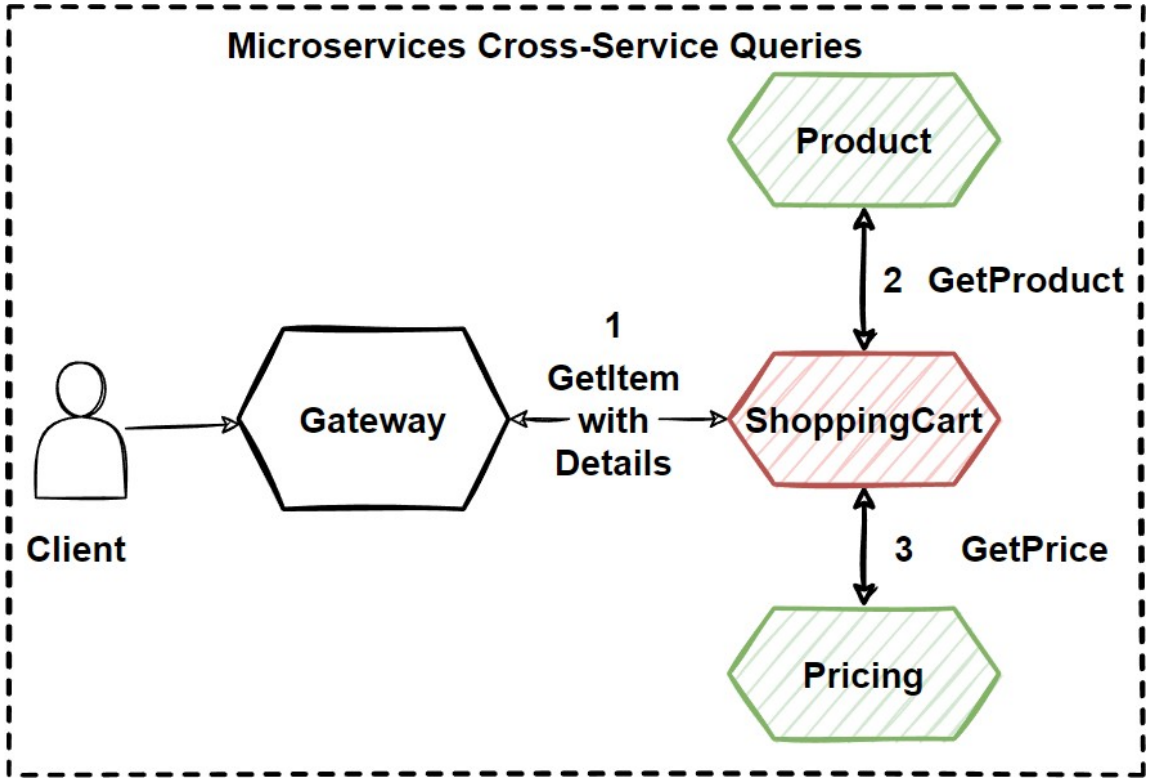




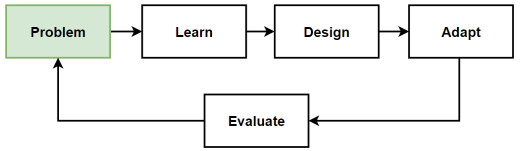
Cross-Service Query Solutions in Microservices



- Perform **queries across microservices** database level in data stores.
- **API Gateway patterns**, we can solve cross-queries with applying Service Aggregator Pattern with create new Aggregator microservices.
- How to **solve problem** with **database level** in data stores?
- How **get item info** from the **user's shopping cart** with get data **Catalog** and **Pricing microservice** ?
- Don’t want to interact with **direct HTTP calls** to get data from other catalog and pricing microservices.
- **Direct synchronous** HTTP calls **makes couple** microservices together and reduce independency of microservices and makes chatty communications.



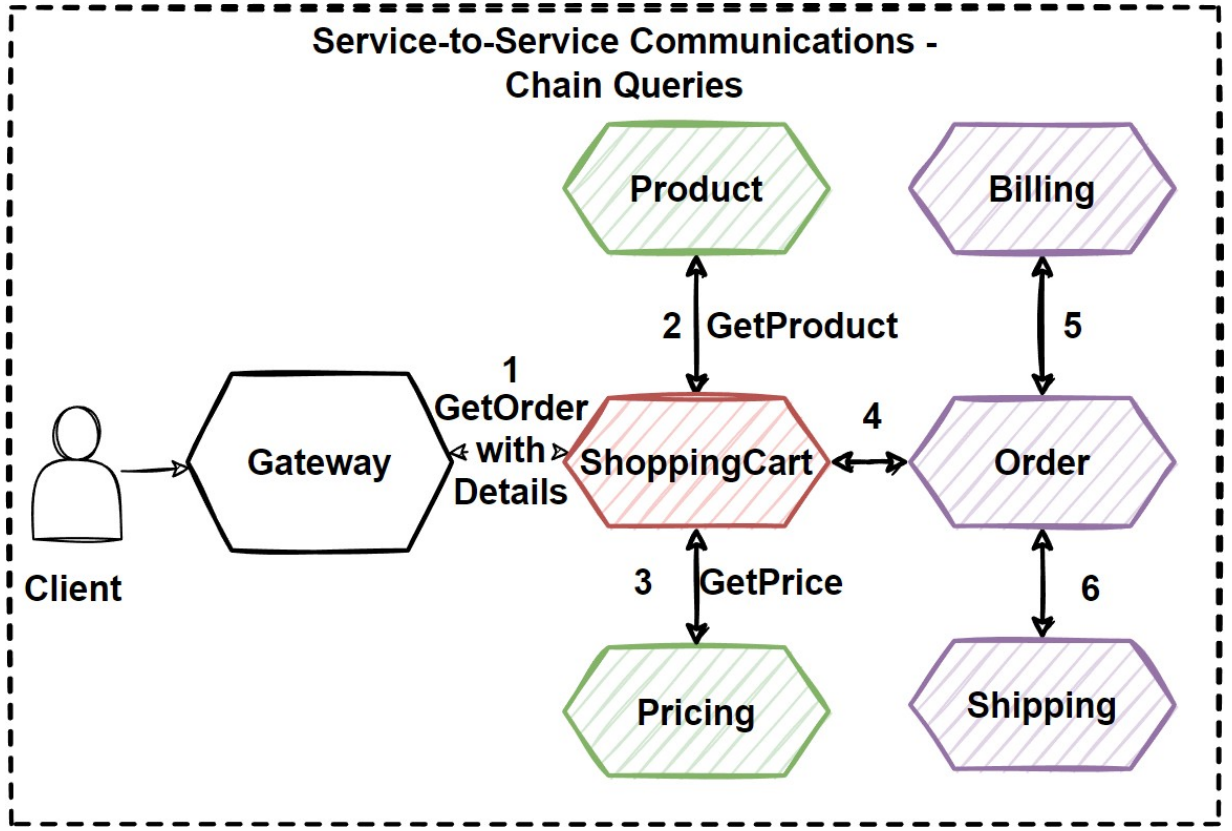
Problem: Cross-Service Queries with Sync Response, Decouple Way and Low Latency



Problems

- Cross-Service Queries with Complex JOINs
- Return Sync Response with low latency
- Provide loosely coupling with decouple services
- Reduce inter-service communication

SOLVE ALL PROBLEMS AT THE SAME TIME!



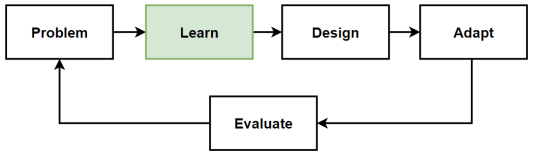
Considerations

- Sync Communication: Use Service Aggregator Pattern but it increase coupling and latency.
- Async Communication: Provide decoupling but query request are waiting immediate response.

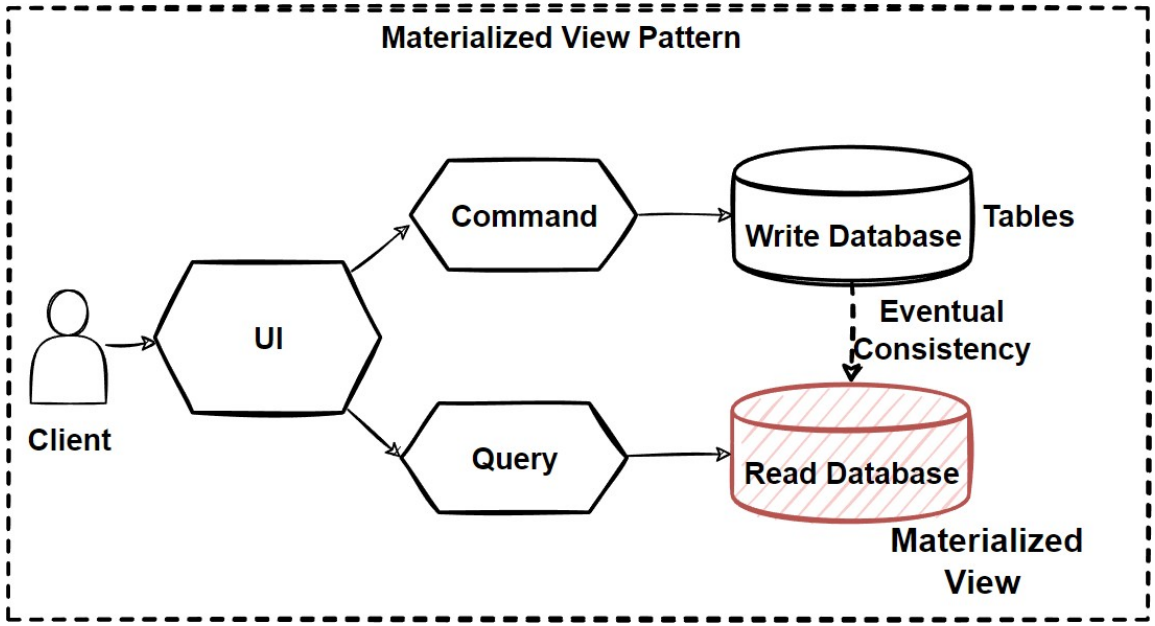
Solutions

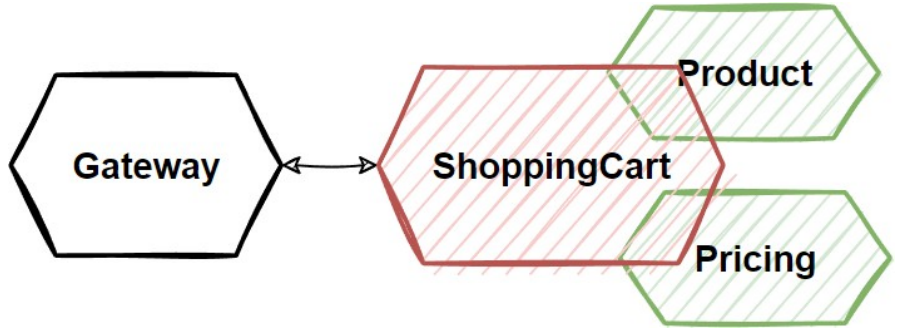
- Materialized View Pattern
- CQRS Design Pattern

Materialized View Pattern

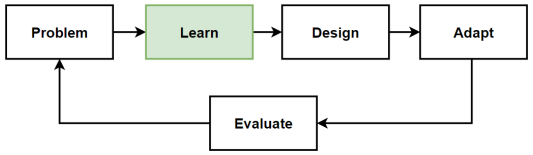


- **Why** do we need **Materialized View Pattern** ?
- Mostly we design our systems with **focusing on databases** and **tables** for managing **data size** and **data integrity**.
- **How** the **data is stored** ? Instead, **how data is read** ?
- **Negative effect** on **queries** on our system.
- **Materialized View Pattern** is **store its own local** and **denormalized copy of data** in the microservice's database.
- Shopping cart service should have table **contains a denormalized copy of the data** from the product and pricing microservices.
- **Eliminates** the need for **expensive cross-service calls**.
- **Reduces coupling** and **improves reliability** and response time with reducing latency.
- Can **execute** the **entire operation** with a **single process**.

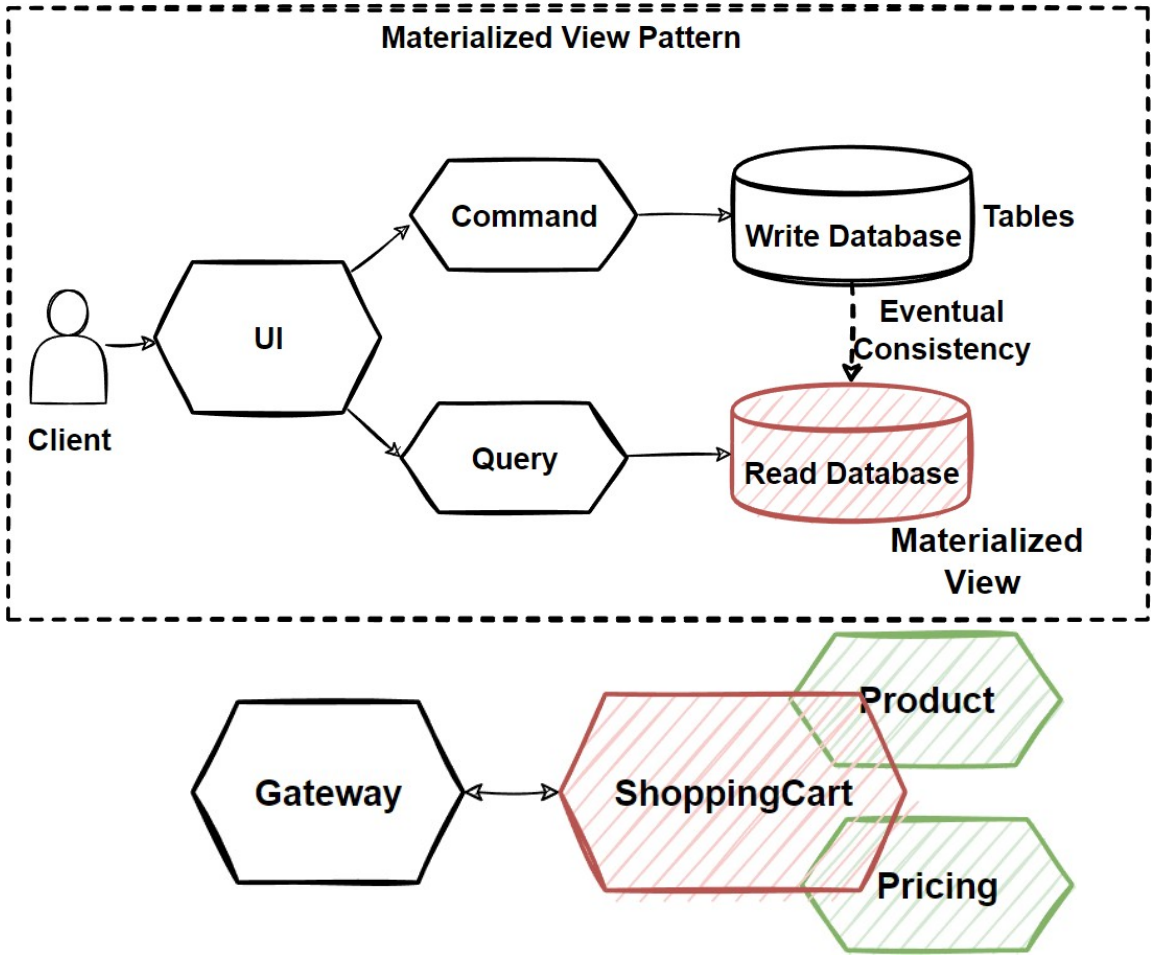




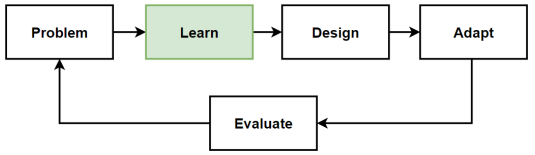
Materialized View Pattern - 2



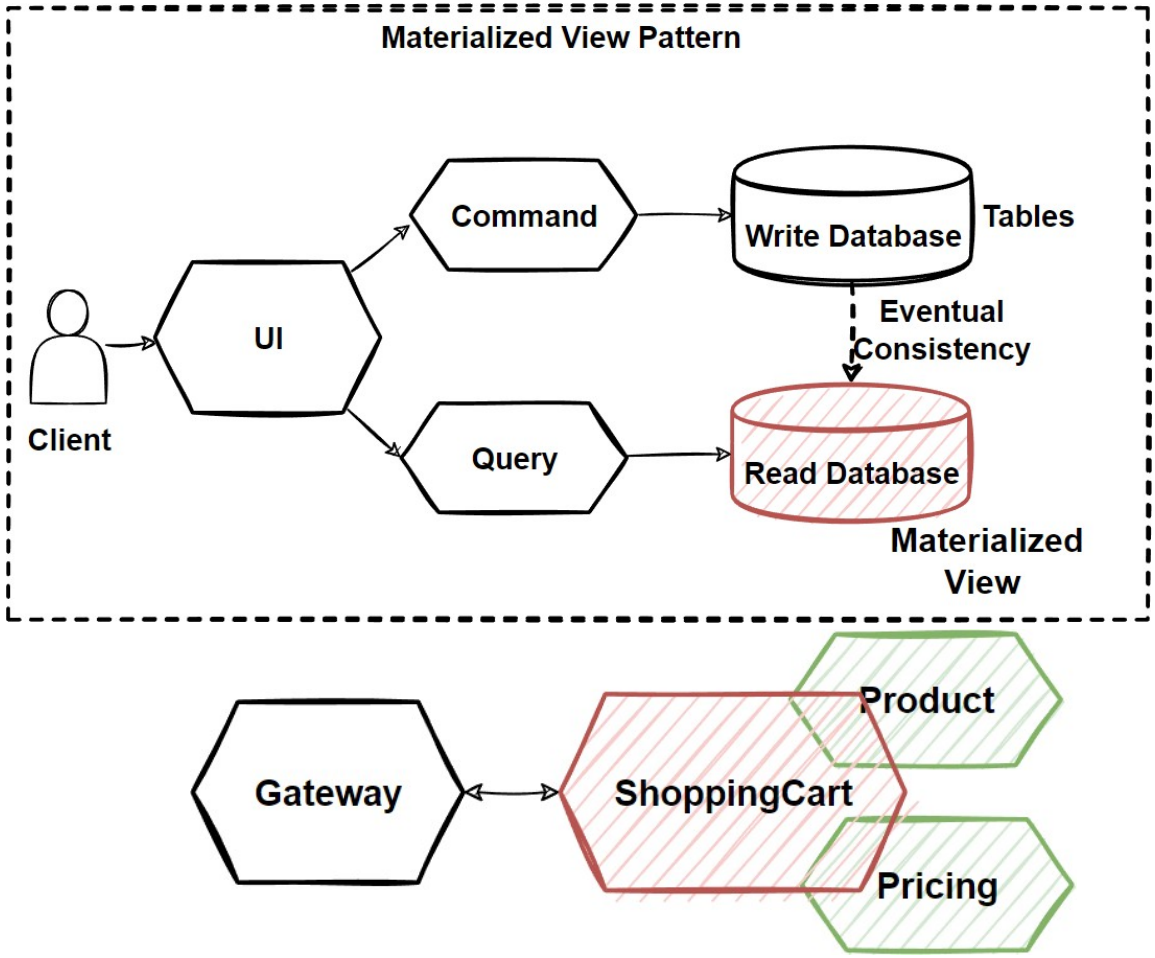
- Also called this local copy of data as a **Read Model**.
- **Instead of querying** the Product Catalog and Pricing services, it maintains **its own local copy** of that **data**.
- Makes SC microservice is **more resilient**.
- **If one** of the **service is down**, then the **whole operation** could be **block** or **rollback**.
- With **Materialized View Pattern**, even if the Catalog and Pricing services are down, **Shopping Cart can continue**.
- **Broke the direct dependency** of other microservices and make faster the response time, help efficient querying and improve application performance.
- **Generate pre-populated views of data**, more suitable format for querying and provide good query performance.
- Includes **joining tables** and **combining data entities** and calculated columns and execute transforms.
- **Views are disposible** and can **rebuilt** from the source.



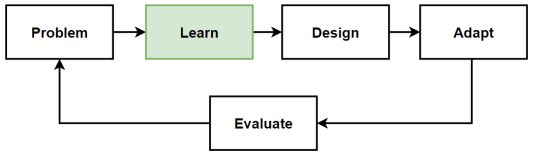
Considerations of Materialized View Pattern



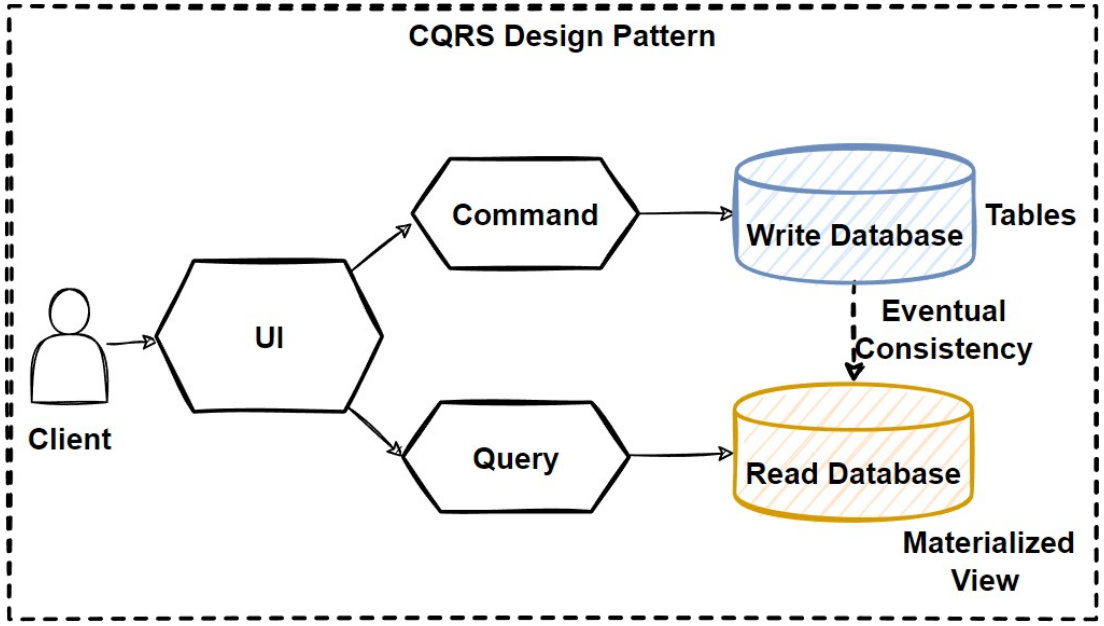
- With appying **Materialized View Pattern**, we have **duplicated data** into our system.
- **Duplicating data** is **not a anti-pattern**, have **strategically duplicating** our data for microservice communications.
- Only one service can be a data ownership.
- How and when the **denormalized data** will be **updated** ?
- **When** the **original data changes** it **should update** into sc microservices.
- **Need to synchronize** the **read models** when the main service of data is **updated**.
- Solve with using **asynchronous messaging** and **publish/subscribe pattern.**
- **Publish an event** and **consumes** from the **subscriber** service to **update** its **denormalized table**.
- Using a **scheduled task**, an **external trigger**, or a manual action to regenerate the table.



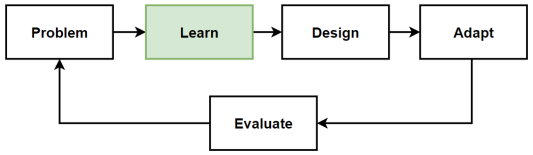
CQRS - Command Query Responsibility Segregation



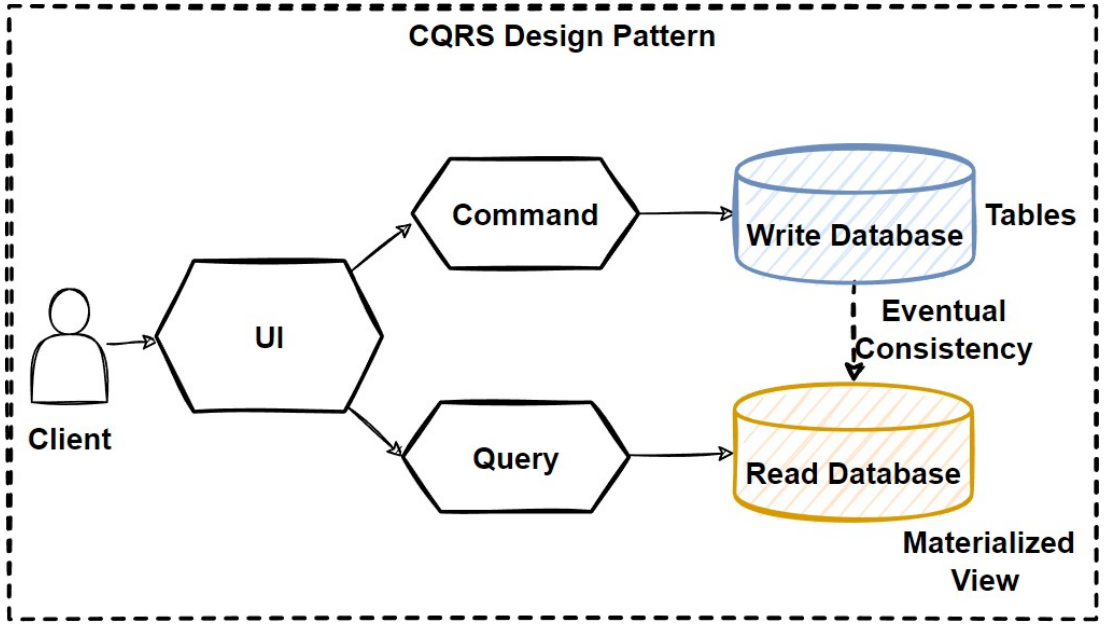
- **CQRS design pattern** in order to avoid **complex queries** to get rid of **inefficient joins**.
- **Separates read** and **write operations** with separating databases.
- **Commands:** changing the state of data into application.
- **Queries:** handling complex join operations and returning a result and don't change the state of data into application.
- Large-scaled **microservices architectures** needs to manage **high-volume data requirements**.
- **Single database** for services can **cause bottlenecks**.
- Uses both **CQRS** and **Event Sourcing** patterns to improve application performance.
- **CQRS** offers to **separates read** and **write data** that provide to maximize query performance and scalability.



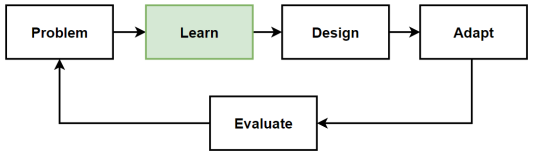
CQRS – Read and Write Operations



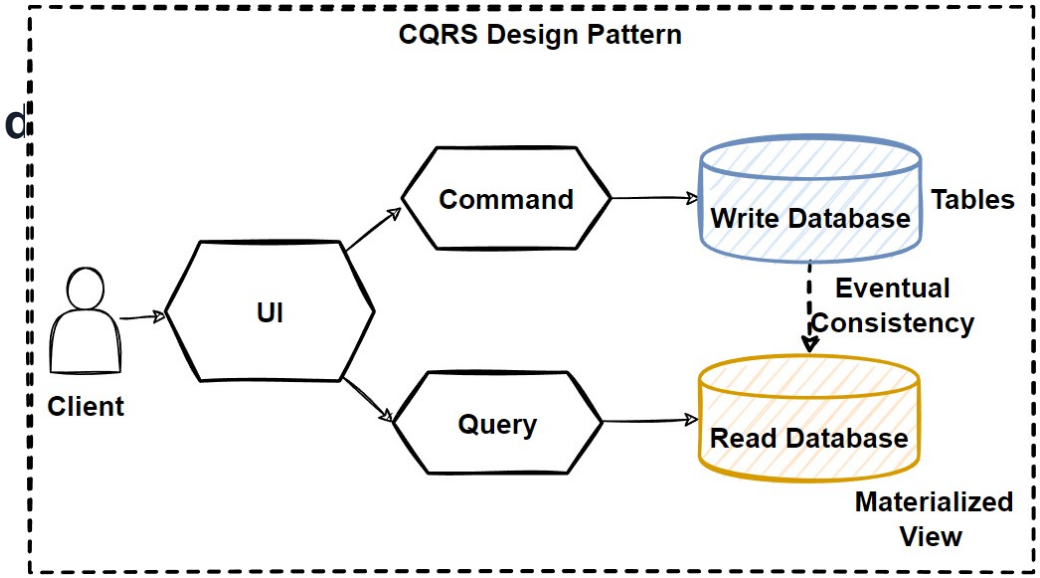
- **Monolithic** has **single database** is both working for **complex join queries**, and also perform **CRUD operations**.
- When **application** goes **more complex**, this **query** and **CRUD** operations will become **un-manageable situation**.
- Application required some **query** that **needs** to **join** more than **10 table**, will **lock** the **database** due to **latency** of **query computation**.
- Performing **CRUD operations** need to make **complex validations** and **process long business logics**, will cause to **lock database** operations.
- **Reading** and **writing database** has different approaches, define different strategy.
- «**Separation of concerns**» principles: separate reading database and the writing database with 2 database.
  - **Read database** uses No-SQL databases with denormalized data.
  - **Write database** uses Relational databases with fully normalized and supports strong data consistency.



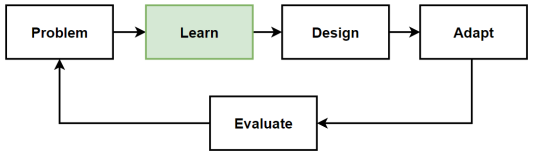
CQRS – Read and Write Operations - 2



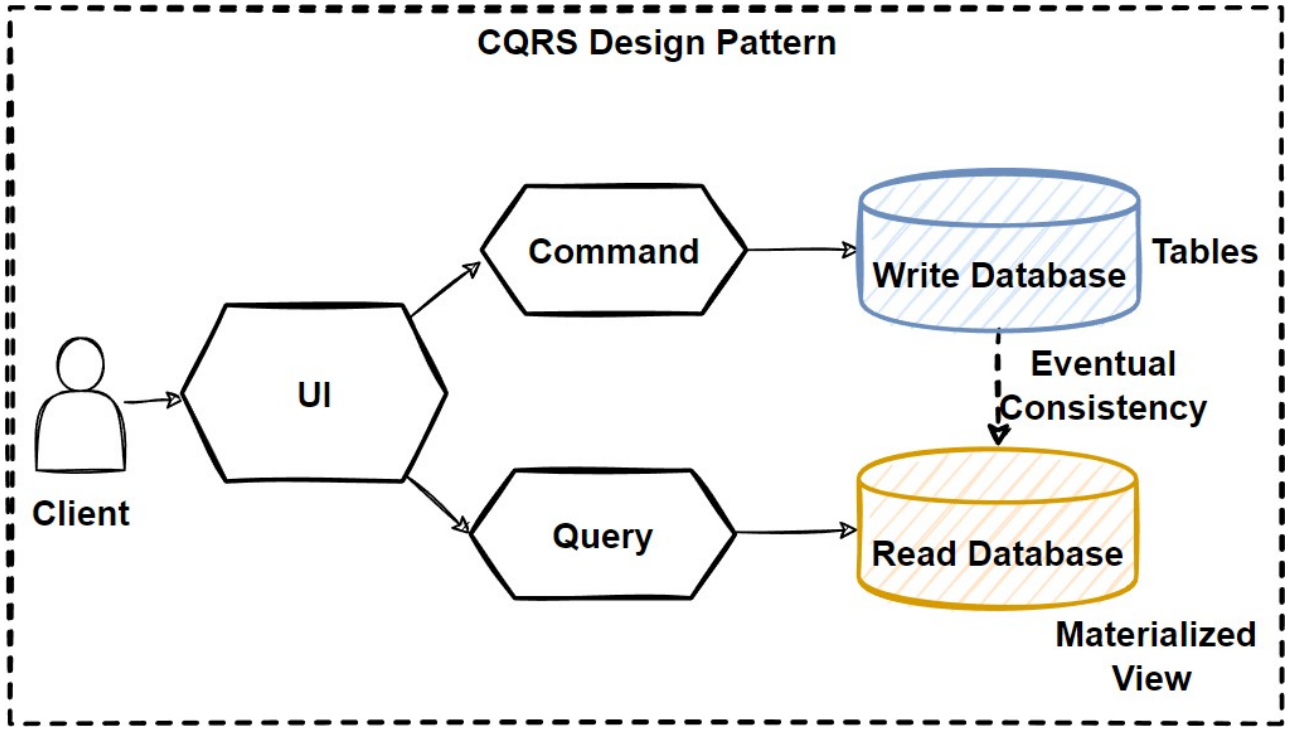
- If our **application** is **mostly reading use cases** and not writing so much, it is **read-incentive application**.
- **Read** and **write operations** are **asymmetrical** and has very different performance and scale requirements.
- To **improve query performance**, the read operation perform **queries** from a **highly denormalized materialized views** to **avoid** expensive repetitive **table joins** and **table locks**.
- **Write operation** which is **command operation**, can perform commands into separate **fully normalized relational database**.
- Supporting **ACID transactions** and **strong data consistency**.
- Commands: **task-based operations** like "add item into shopping cart" or "checkout order".
- **Commands** can be handle with message broker systems that provide to process commands in async way.
- **Queries:** never modify the database, always return the JSON data with DTO objects.



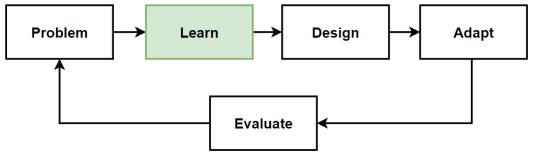
CQRS – Synchronization with Read and Write DB



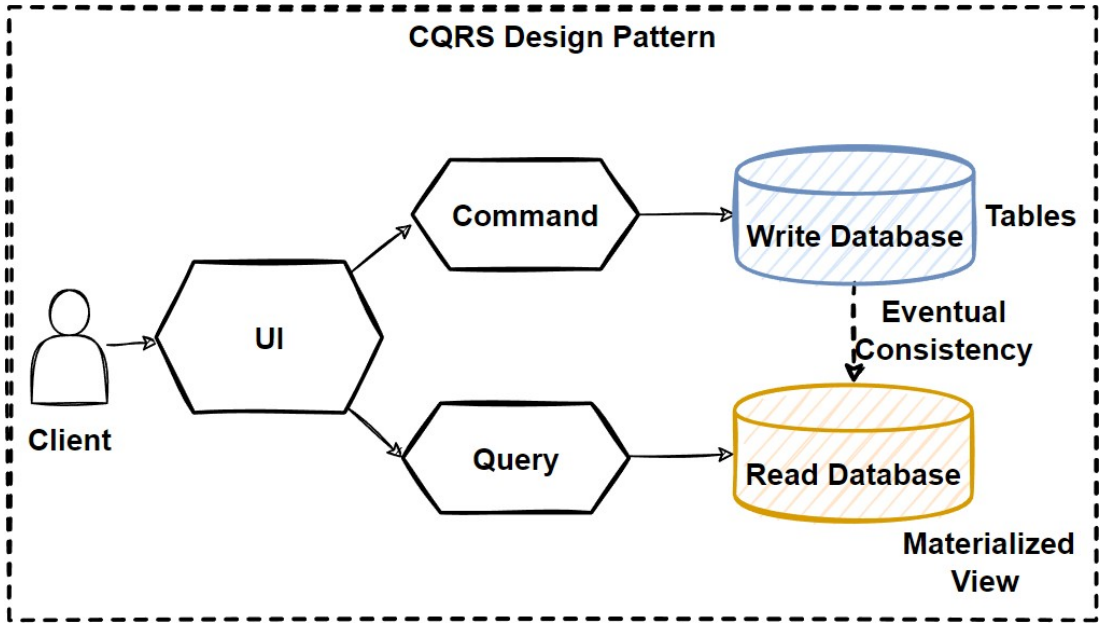
- **Keep sync** both **read** and **write databases**.
- **Publishes an event t**hat subscribe from **Read Database** and **update** the **read table** accordingly.
- **Synchronization** handles with async communication using message brokers.
- This creates **Eventual Consistency principle**.
- The **Read** database **eventually synchronizes** from the **Write** database.
- **Some lag** in the process due to **async communication** with message brokers that applies publish/subscribe pattern.
- Welcome to **Eventual consistency.**



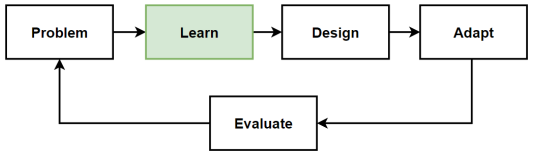
Benefits of CQRS



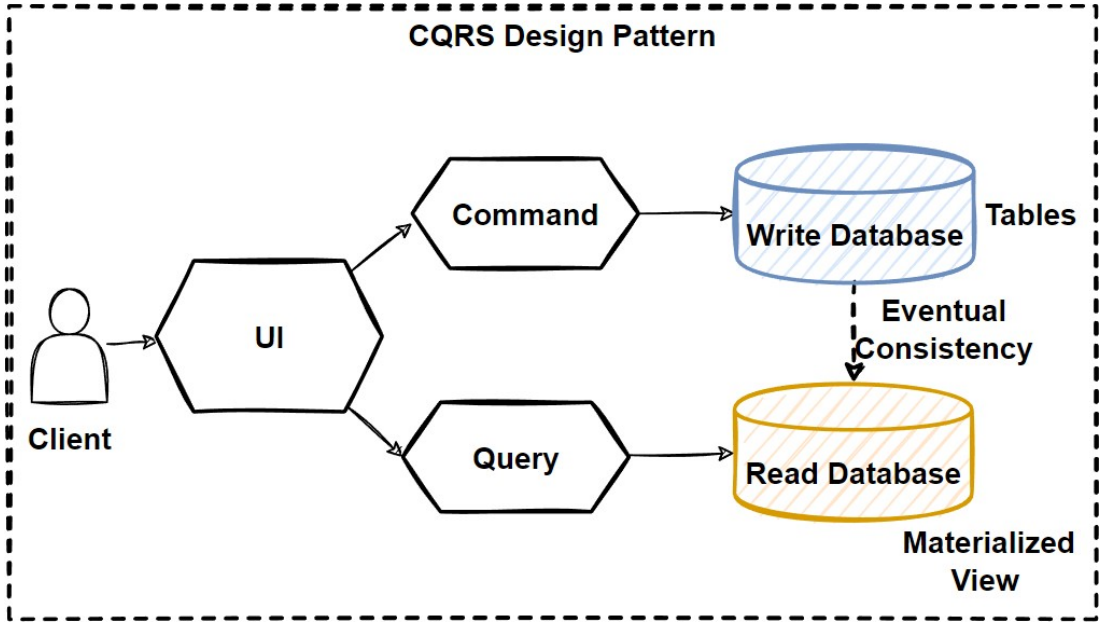
- **Scalability**
  When we separate Read and Write databases, we can also scale these separate databases independently. Read databases follows denormalized data to perform complex join queries.
- If application is read-incentive application, we can scale read database more then write database.
- **Query Performance**
  The read database includes denormalized data that reduce to comlex and long-running join queries. Complex business logic goes into the write database. Improves application performance for all aspects.
- **Maintability and Flexibility**
  Flexibility of system that is better evolve over time and not affected to update commands and schema changes by separating read and write concerns into different databases.
- Better implemented if we physically separate the read and write databases.



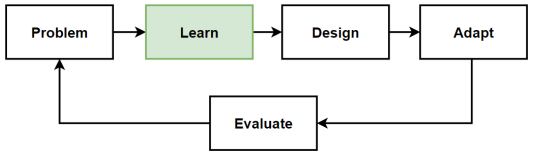
Drawbacks of CQRS



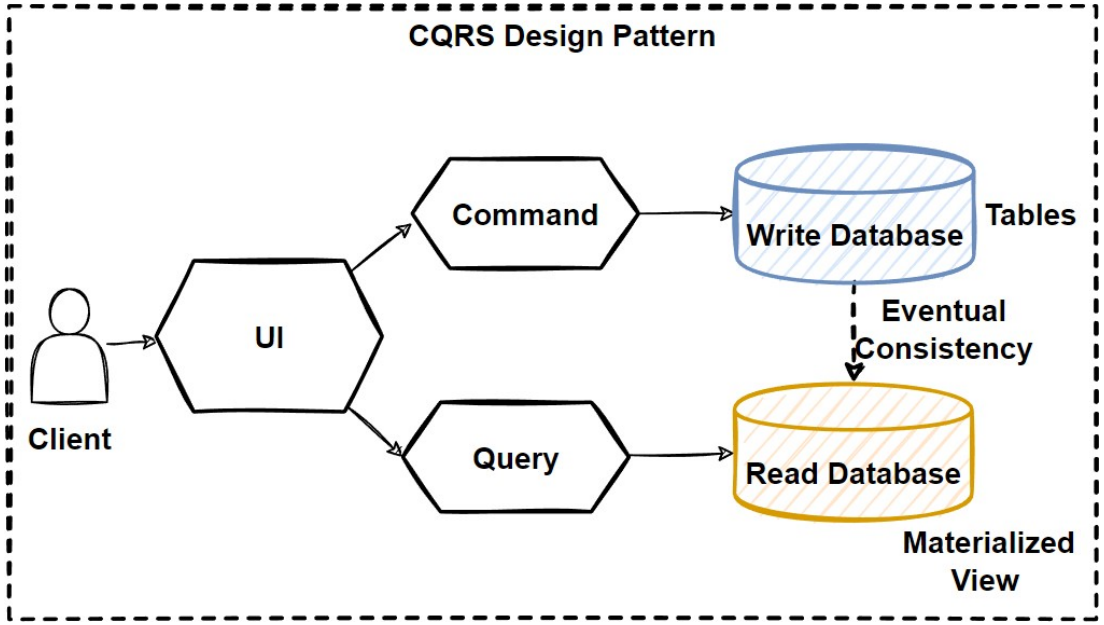
- **Complexity**
  CQRS makes your system more complex design.
  Strategically choose where we use and how we can separate read and write database.
- **Eventual Consistency**
  The read data may be stay old and not-updated for a particular time. So the client could see old data even write database updated, it will take some time to update read data due to publish/subscribe mechanism.
- We should **embrace** the **Eventual Consistency** when using **CQRS**, if your application **required strong consistency** than **CQRS is not good to apply**.



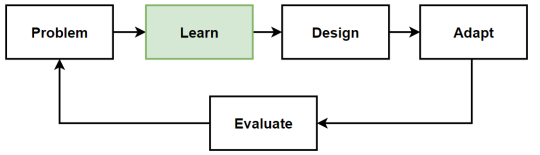
Best Practices for CQRS



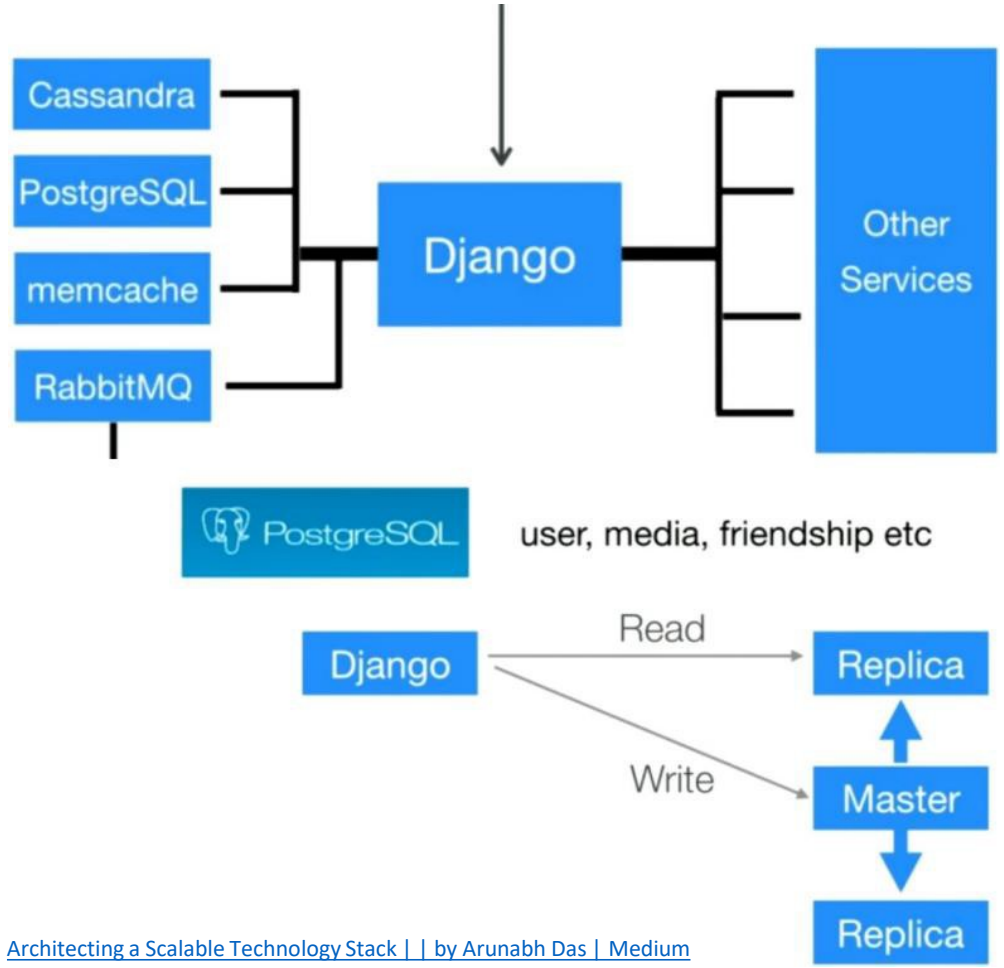
- **Best practices** to **separate read** and **write database** with 2 database **physically**.
- **Read-intensive** that means **reading more than writing**, can define **custom data schema** to **optimized for queries**.
- **Materialized View Pattern** is good example to implement reading databases.
- **Avoid complex joins** and mappings with **pre-defined** fine-grained **data** for **query operations**.
- **Use different database** for reading and writing database types.
- Using **No-SQL** document database **for reading** and using **Relational** database **for CRUD** operations.



Instagram Database Architecture with CQRS

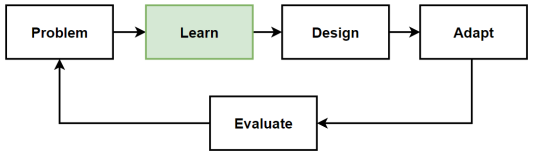


- Uses **two database systems** for different use cases:
- **Relational database - PostgreSQL** and the other is **No-SQL database – Cassandra.**
- Uses **No-SQL Cassandra** database for user stories.
- Uses **Relational PostgreSQL database** for User Information bio update.

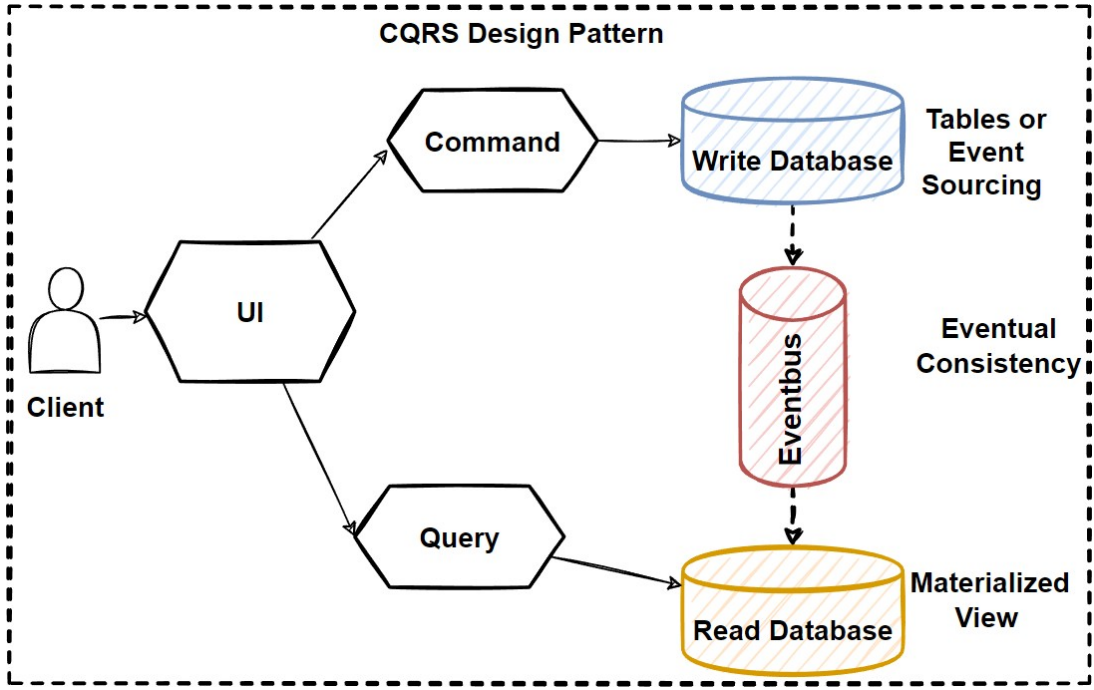


Architecting a Scalable Technology Stack | | by Arunabh Das | Medium

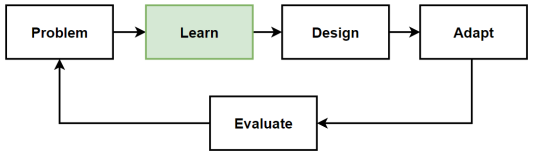
How to Sync Read and Write Databases in CQRS ?



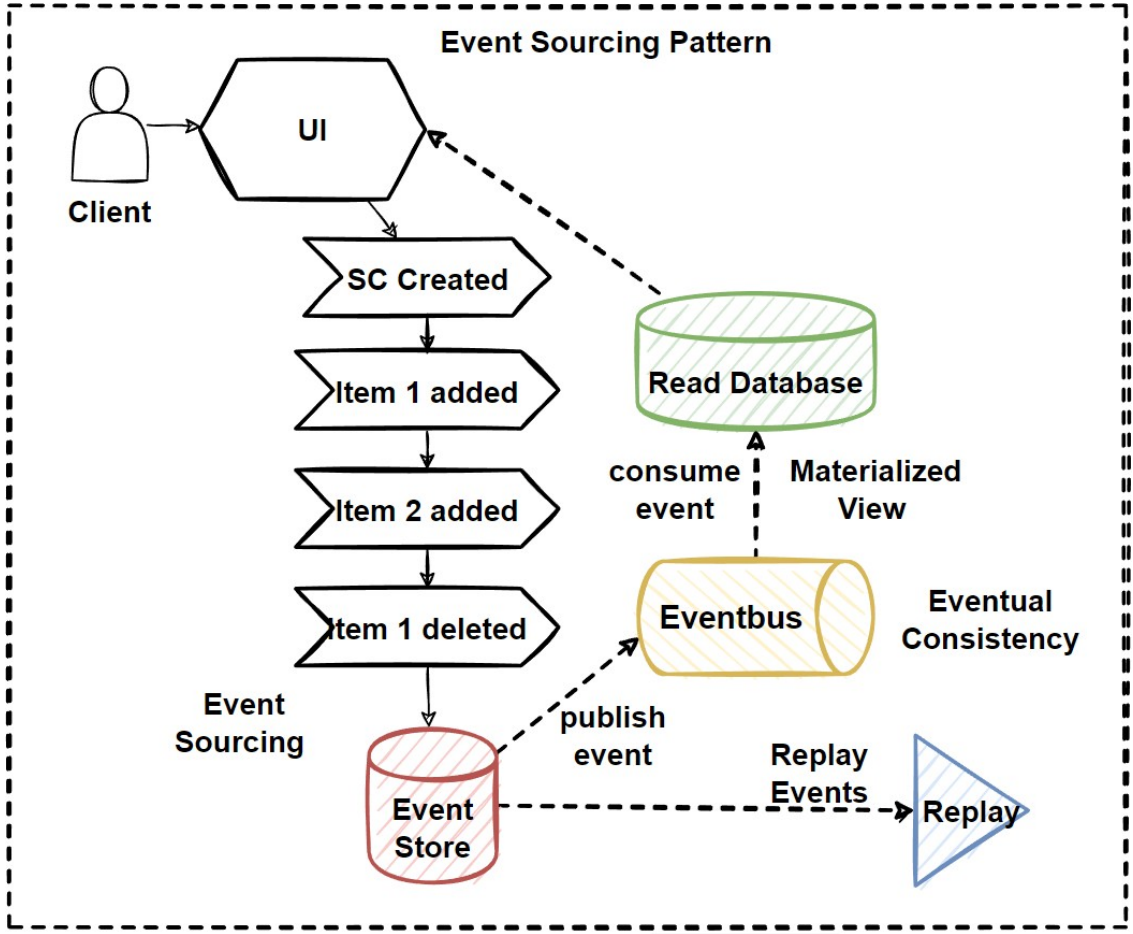
- **Event-Driven Architecture**, when something update in write database, **publish an update event** with using message broker systems, **consume by the read database** and **sync data** according to latest changes.
- Creates a **consistency issue**, the **data** would not be **reflected immediately** due to async communication with message brokers.
- «**Eventual Consistency**» The read database **eventually synchronizes** with the write database, and take some time to update read database in the async process.
- Take read database from replicas of write database. applying **Materialized View Pattern** can significantly increase query performance.
- **Event Sourcing Pattern** is the first pattern we should consider to use with CQRS.
- **CQRS** is using with "**Event Sourcing Pattern**" in Event-Driven Architectures.



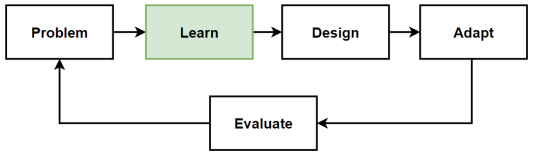
Event Sourcing Pattern



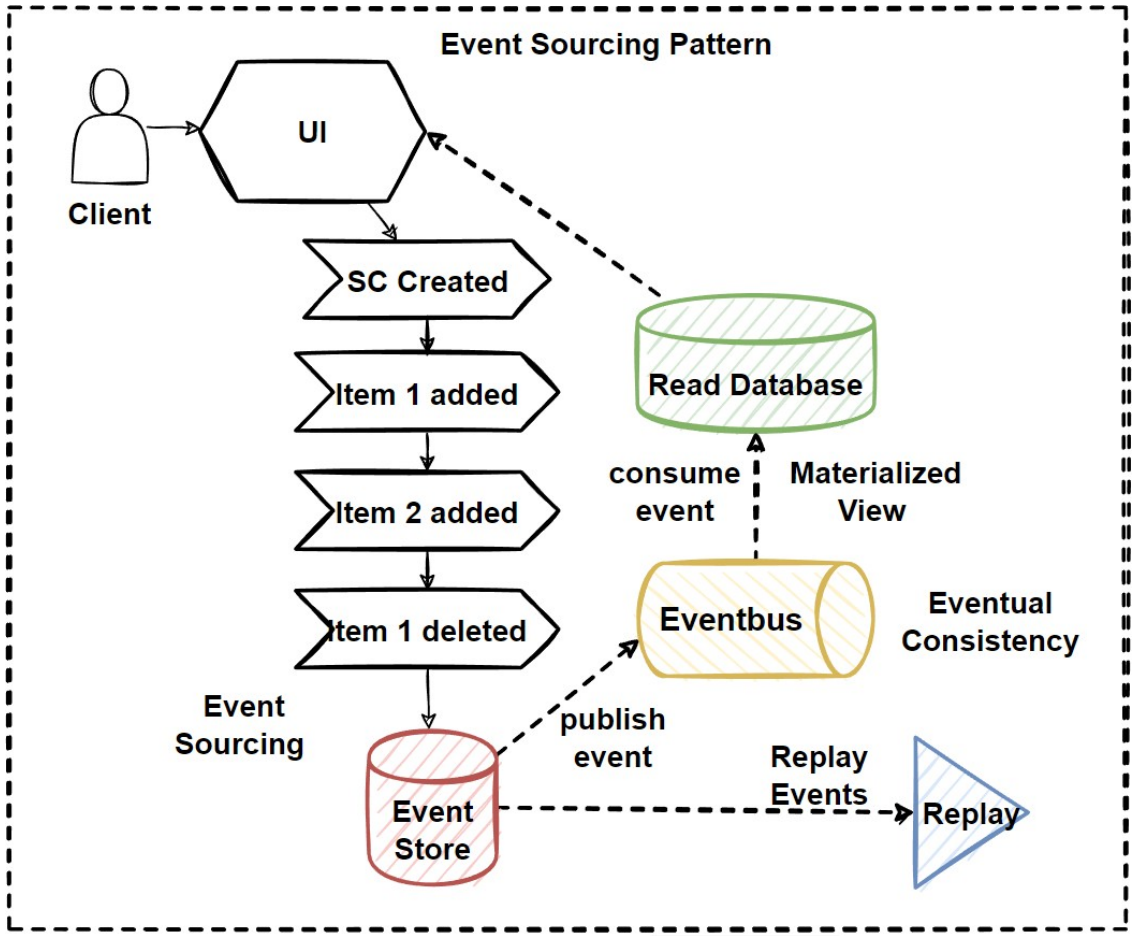
- Most applications **saves data** into databases with **the current state of the entity**. I.e. user change the email address table, email field updated with the latest updated one. Always know the **latest status** of the **data**.
- In large-scaled architectures, **frequent update database** operations can **negatively impact database performance**, **responsiveness**, and **limits** of **scalability**.
- **Event Sourcing pattern** offers to persist each action that affects to data into **Event Store database**. And call all these **action** as a **event**.
- **Instead of saving latest status** of data into database, Event Sourcing pattern offers to **save all events into database** with **sequential ordered** of data events.
- This **events database** called **Event Store**.



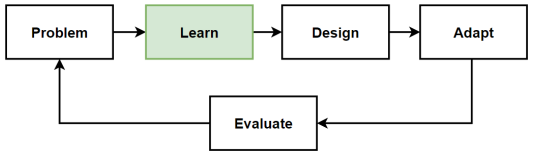
Event Sourcing Pattern - 2



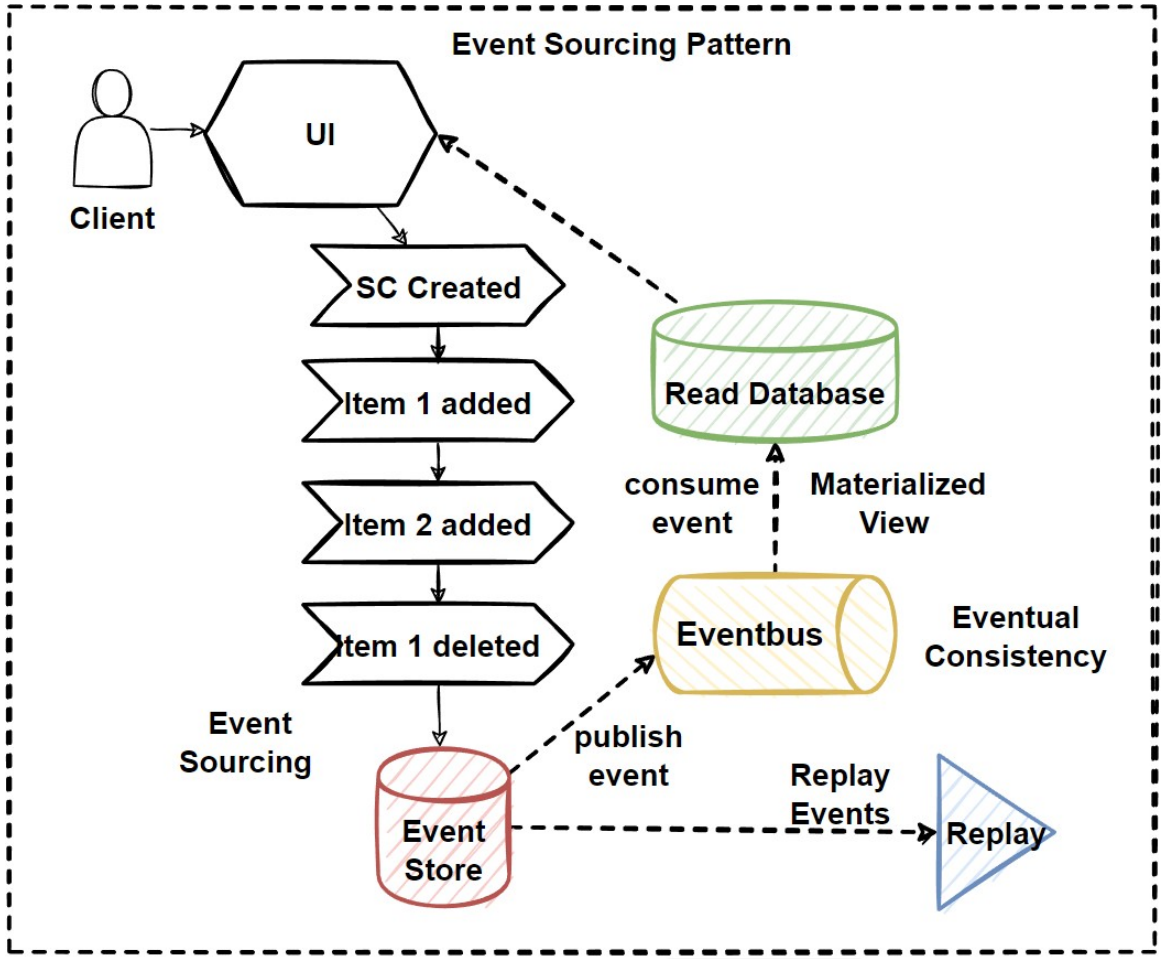
- **Instead of overriding** the **data** into table, It **create a new record** for each change to data, and it becomes **sequential list** of past events.
- **Event Store database** become the **source-of-truth** of **data**.
- **Sequential event list** using for **generating Materialized Views** that represents **final state** of **data** to perform queries.
- Event Store **convert to read database** with following the **Materialized Views Pattern**.
- Convert operation can handle by **publish/subscribe pattern** with **publish event** with message broker systems.
- Event list gives ability to **replay events** at **given certain timestamp.**
- It is able to **re-build latest status** of data with **replaying events**.



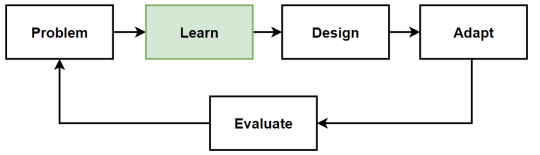
CQRS with Event Sourcing Pattern

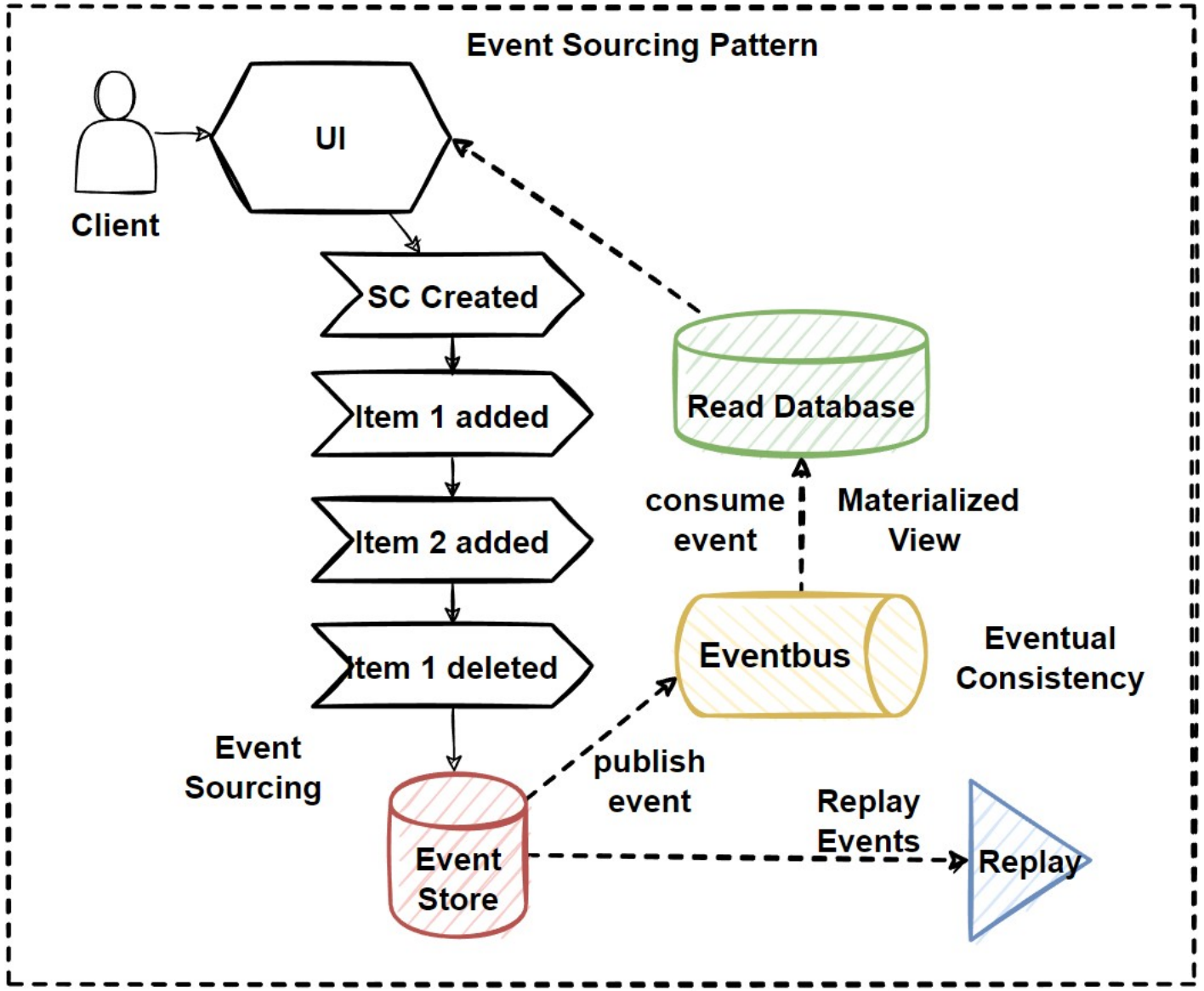


- **CQRS** pattern is mostly **using with** the **Event Sourcing pattern.**
- **Store events** into the **write database**; **source-of-truth events** database.
- **Read database** of CQRS pattern provides **materialized views** of the data with **denormalized tables**.
- **Materialized views** read database **consumes events** from write database **convert them** into **denormalized views**.
- The **writing database** is **never save status** of data only events actions are stored.
- **Store history of data** and able to **reply any point of time** in order to **re-generate status** of data.
- System can **increased query performance** and **scale databases independently.**

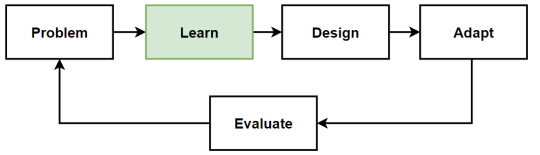


CQRS with Event Sourcing Pattern

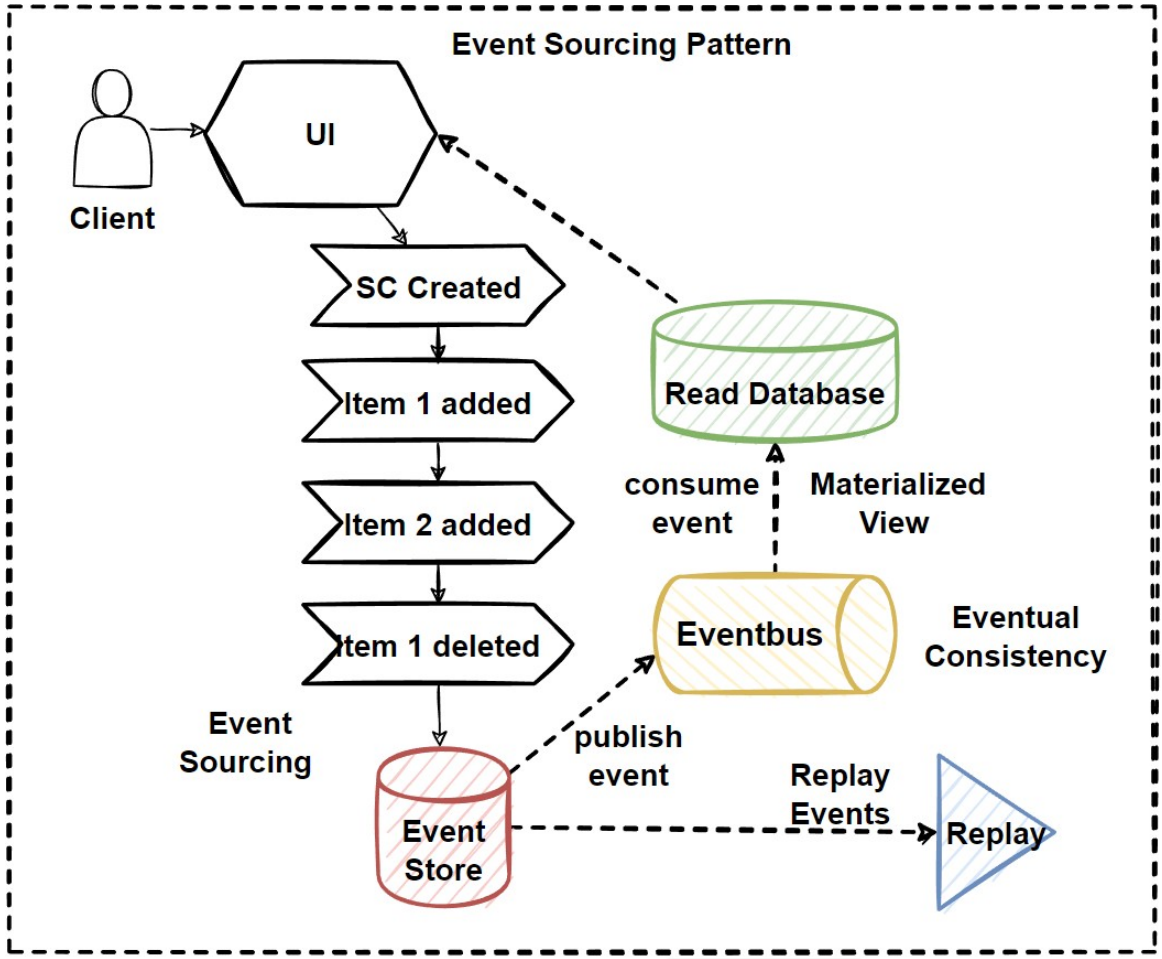




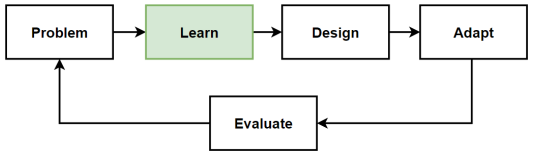
Eventual Consistency Principle



- **CQRS** with **Event Sourcing Pattern** leads **Eventual Consistency.**
- **Eventual Consistency** is especially used for systems that **prefer high availability** to **strong consistency**.
- The system will **become consistent after a certain time**.
- We called this latency is a **Eventual Consistency** Principle and offers to be **consistent after a certain time**.
- There are **2 type of "Consistency Level":**
- **Strict Consistency:** When we save data, the data should affect and seen immediately for every client.
- **Eventual Consistency:** When we write any data, it will take some time for clients reading the data.



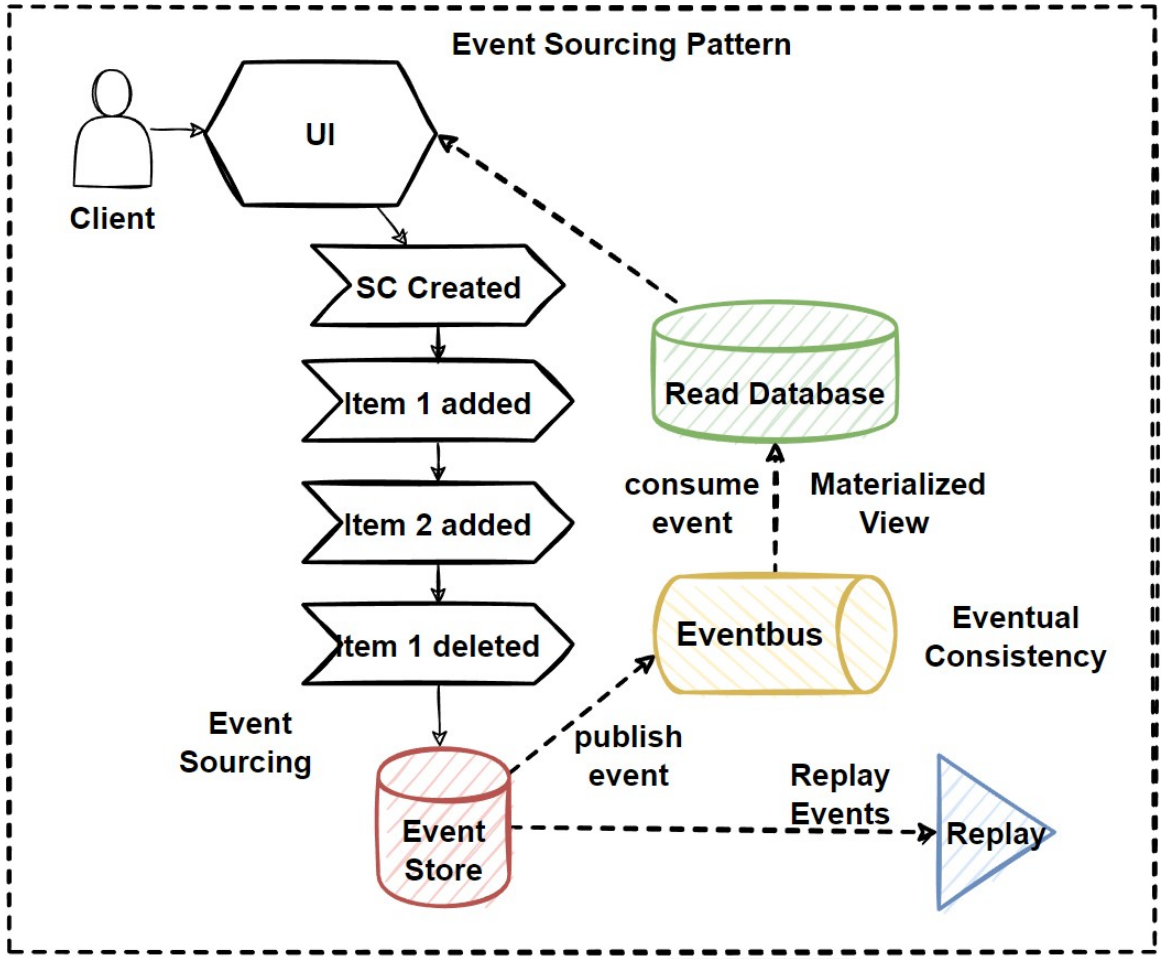
Eventual Consistency Principle - 2



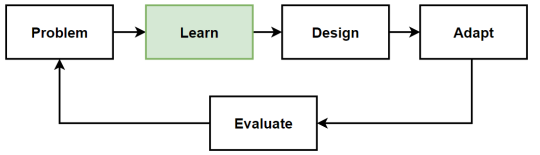
CQRS Design Pattern and **Event Sourcing** patterns:

1. When user perform any action into application, this will save actions **as a event into event store**.
2. Data will **convert to Reading database** with following the publish/subscribe pattern with using message brokers.
3. Data will be **denormalized** into **materialized view database** for querying from the application.

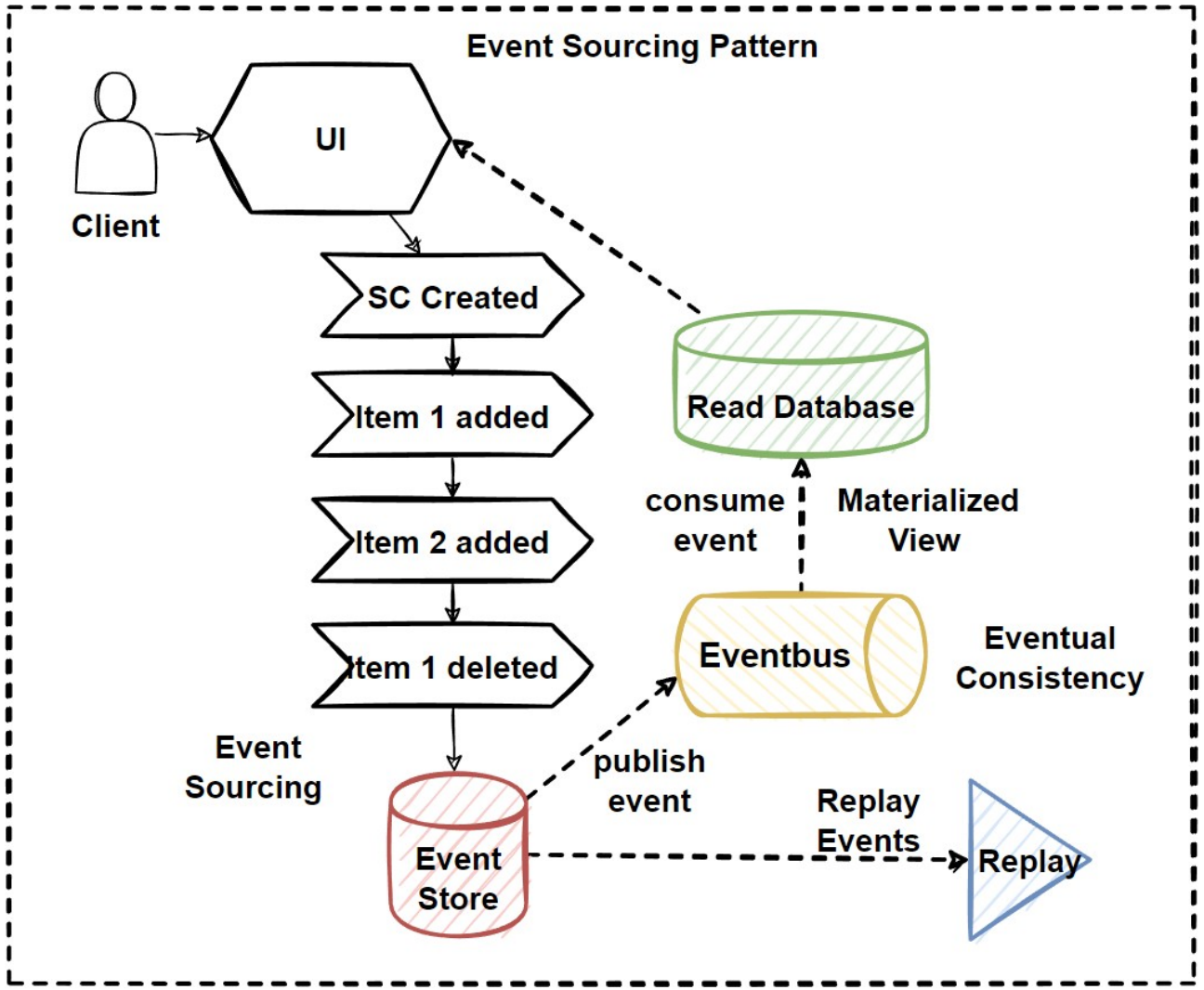
We call this process is a **Eventual Consistency Principle**.



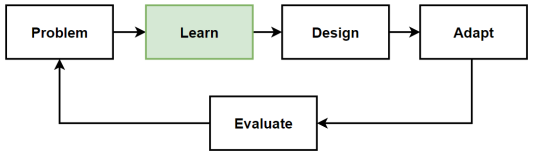
Microservices Data Patterns and Principles



- Materialized View Pattern
- CQRS Design Pattern
- Event Sourcing Pattern
- Eventual Consistency Principle

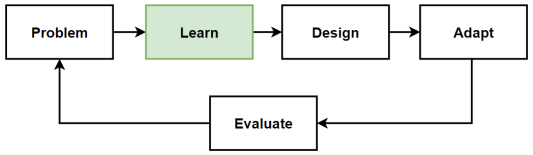


Instagram System Architecture

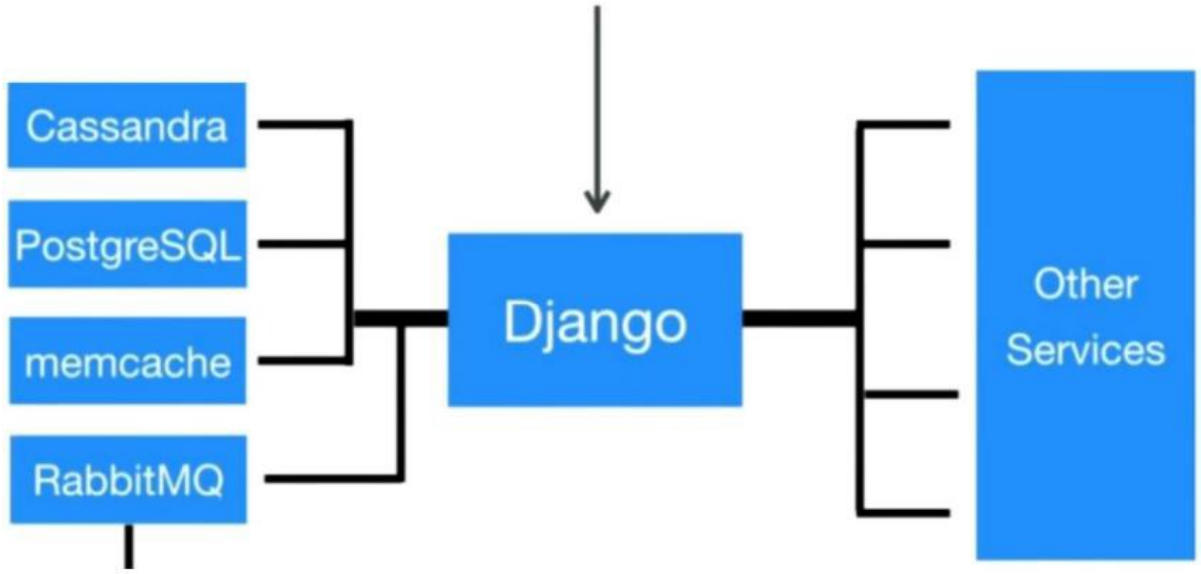


- **Instagram** is an application that is used **all over the world** with **under heavy traffic**.
- Visited by **500 million people**, Almost **100 million photos** are shared, **400 million stories.**
- **Huge amount of data** and **interactions** that need to handle with the strong architecture of Instagram.
- Have **many servers**, data **center to spread** to **many parts** of the world.
- **Network delays** can cause wait for a long time for the **user sending requests.**
- **Instagram** has a **Data Center** in many parts of the world.

Instagram System Architecture - 2

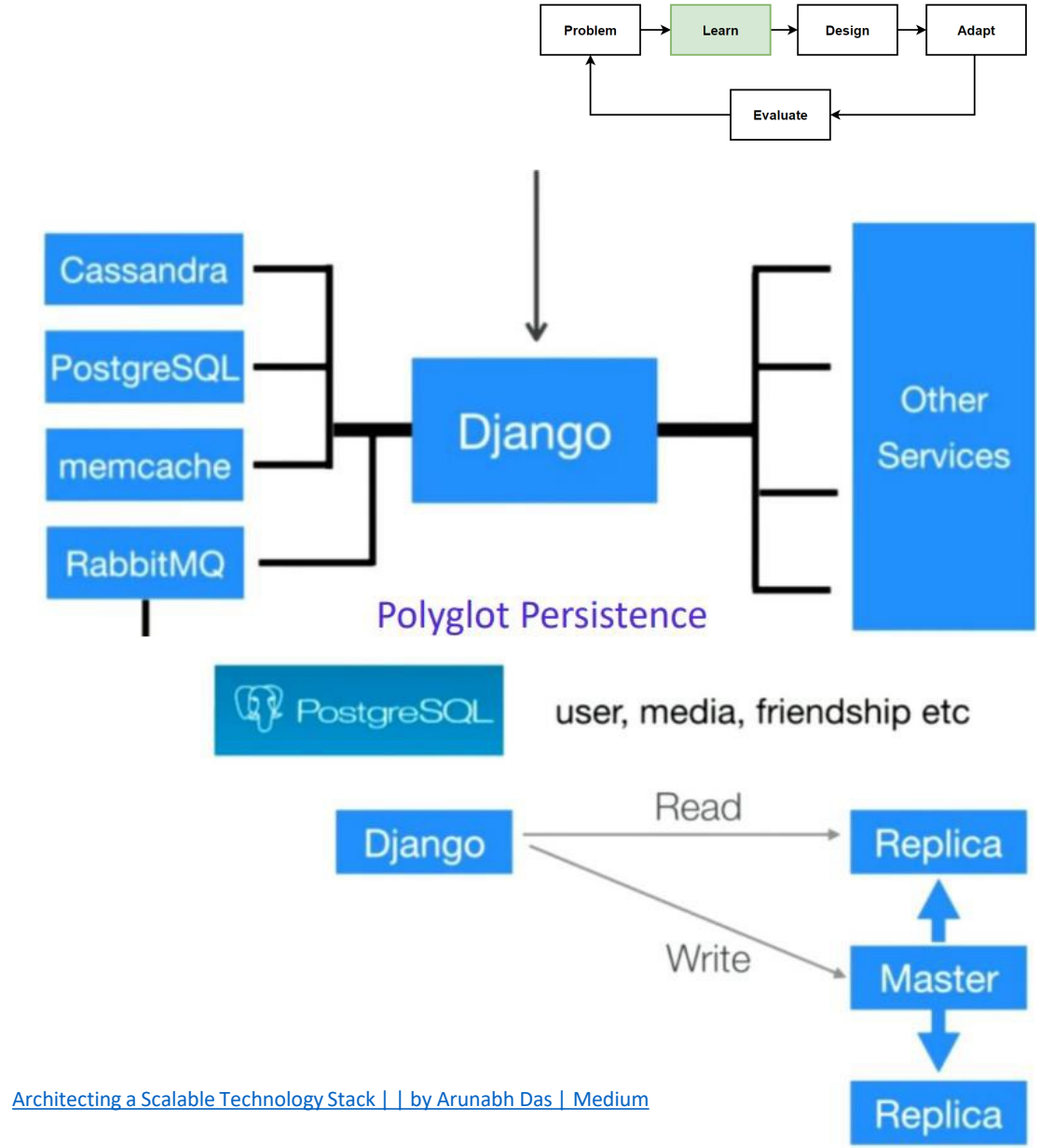


- **Instagram** uses **two database** systems: **PostgreSQL, Cassandra.**
- Designed that users can communicate with **the nearest data center** and **receive quick responses** to requests.
- **CAP Theorem:** either Availability or Consistency must be selected in a distributed system.
- Instagram **decided that Availability** was more important to them and thought that **Eventual Consistency** would be sufficient.
- **Polygot Persistence:** uses relational PostgreSQL and No-SQL Cassandra databases.
- **CQRS Design Pattern:** separated read-incentive use cases with Cassandra No-SQL database which uses is user stories.
- **Eventual Consistency:** Updates reflect with lag, decide the be Availability every time.



Instagram Database Architecture

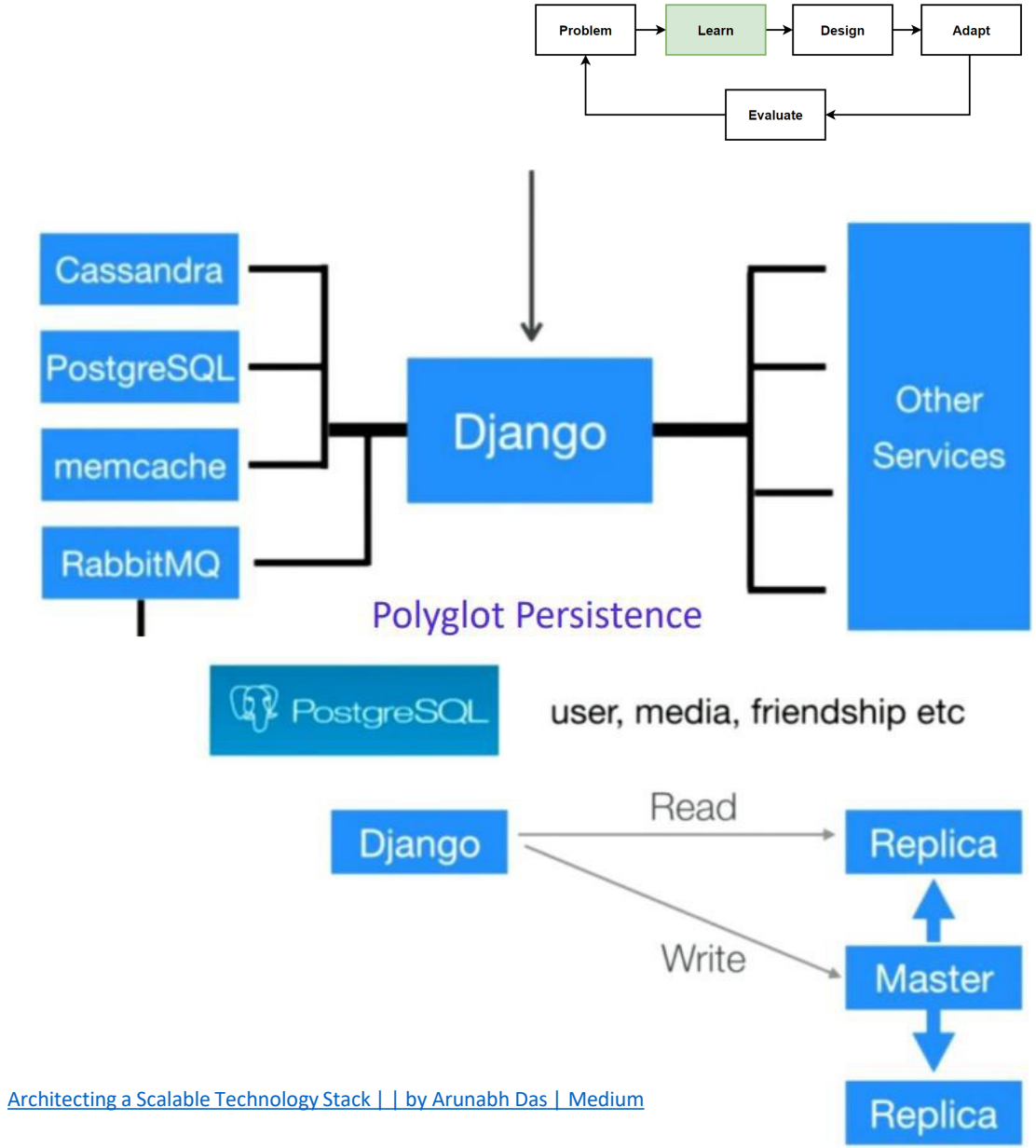
- Instagram uses two database systems: PostgreSQL, Cassandra.
- **Why use PostgreSQL relational database ?**
  PostgreSQL has a "Master-Slave" architecture. PostgreSQL on Instagram is configured to have one Master and multiple read-only Slaves.
- If the transaction causes a change in the data like update or delete user info, that request must be forwarded to the Master.
- Any update that occurs in the master is sent from the master to the slaves, ensuring the synchronization of the databases.
- **What data does Instagram keep in PostgreSQL?**
  user information, user relations (friendship), tags, albums, photos and videos.



Architecting a Scalable Technology Stack | | by Arunabh Das | Medium

Instagram Database Architecture - 2

- **Why use Cassandra No-SQL database ?**
  Cassandra has a Master-Master architecture we can call Master-less architecture. Cassandras communication is multi-directional between Data Centers of Instagram.
- **What data does Instagram keep in Cassandra ?**
  User feed, Counters, Messaging
- The user flow of Instagram is includes the last posts of the people that user follows, resolved by making use of Cassandra.
- **Why was Cassandra chosen ?**
  Cassandra has an auto-sharding feature like many NoSQL databases. Sharding helps keep data divided among nodes.
- Disk usage of the servers are optimized and the workload between the nodes is easily distributed.



Architecting a Scalable Technology Stack | | by Arunabh Das | Medium

Before Design – What we have in our design toolbox ? - Data

Architectures Patterns&Principles

- **Microservices Architecture**
  - The Database-per-Service Pattern
  - Polygot Persistence
  - Decompose services by scalability
  - The Scale Cube
  - Microservices Decomposition Pattern
  - Microservices Communications Patterns
  - **Microservices Data Management Patterns**

Microservices Data Choosing Database

- The Shared Database Anti-pattern
- Relational and NoSQL Databases
- CAP Theorem–Consistency, Availability, Partition Tolerance
- Data Partitioning: Horizontal, Vertical and Functional Data Partitioning
- Database Sharding Pattern

Microservices Data Commands&Queries

- **Materialized View Pattern**
- **CQRS Design Pattern**
- **Event Sourcing Pattern**
- **Eventual Consistency Principle**

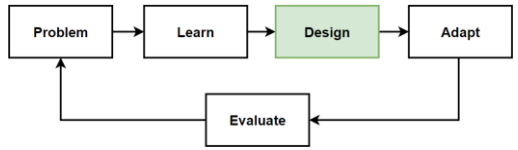
FR

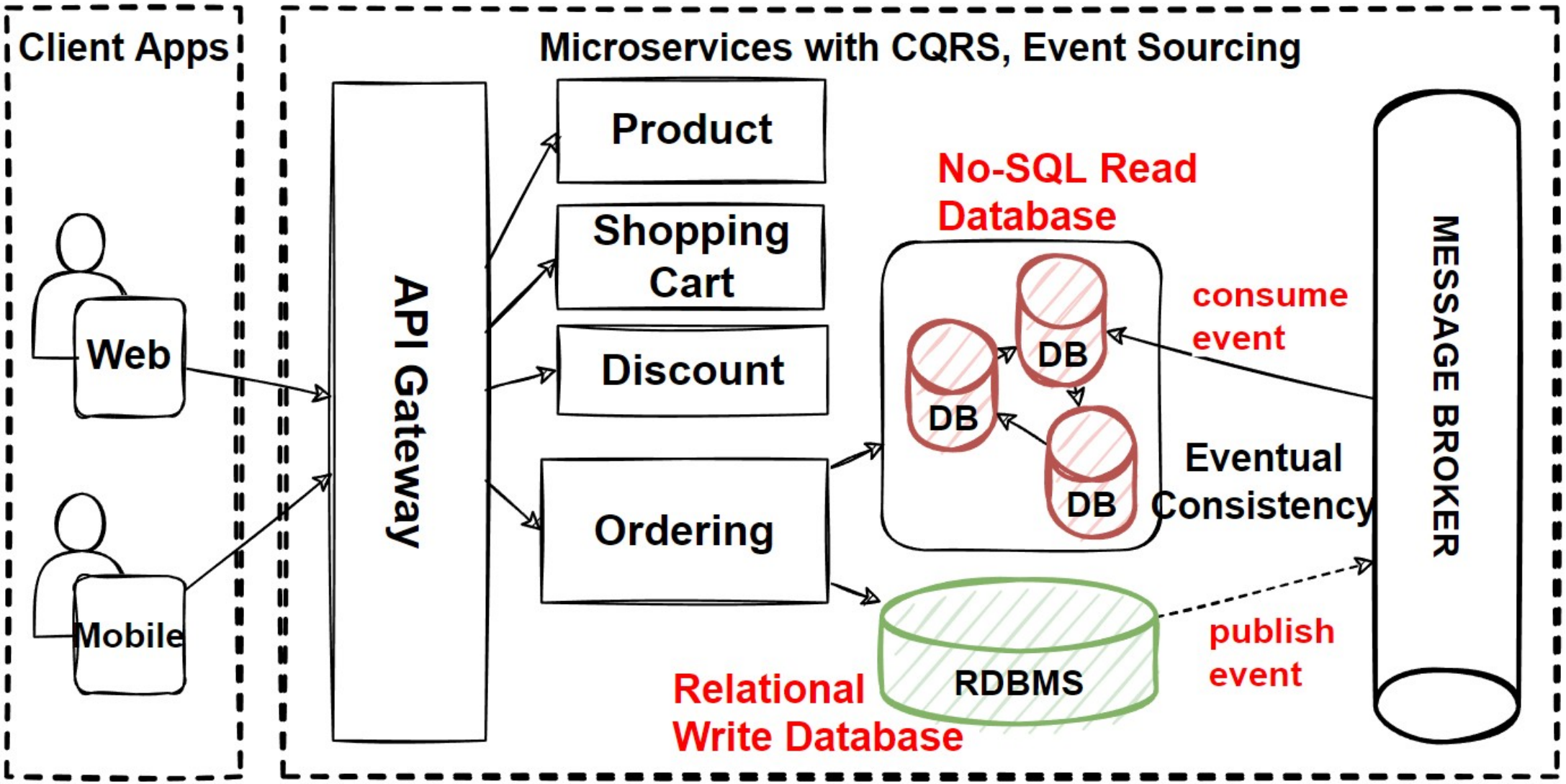
- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an orde
- List my old orders and order items history

Non-FR

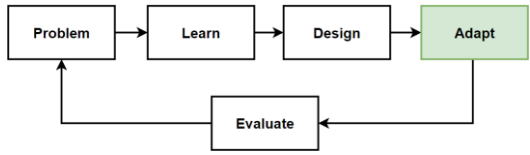
- High Scalability
- High Availability
- Millions of Concurrent User
- Independent

Design: Microservices Architecture with CQRS, Event Sourcing, Eventual Consistency





Adapt: Microservice Architecture with CQRS, Event Sourcing, Eventual Consistency

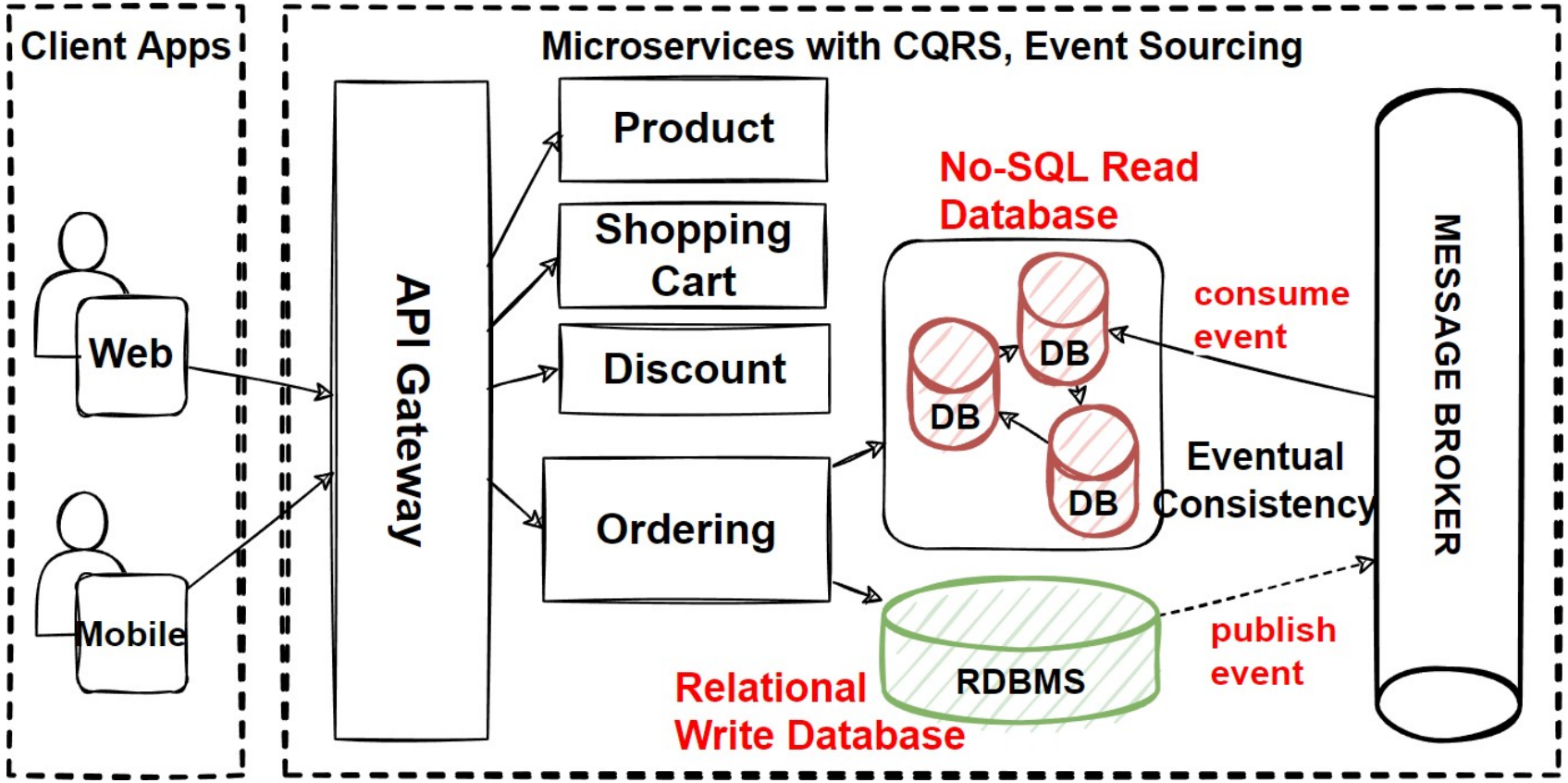


Frontend SPAs
- Angular
- Vue
- React

API Gateways
- Kong Gateway
- Express Gateway
- Amazon AWS API Gateway

Message Brokers
- Kafka
- RabbitMQ
- Amazon EventBridge, SNS

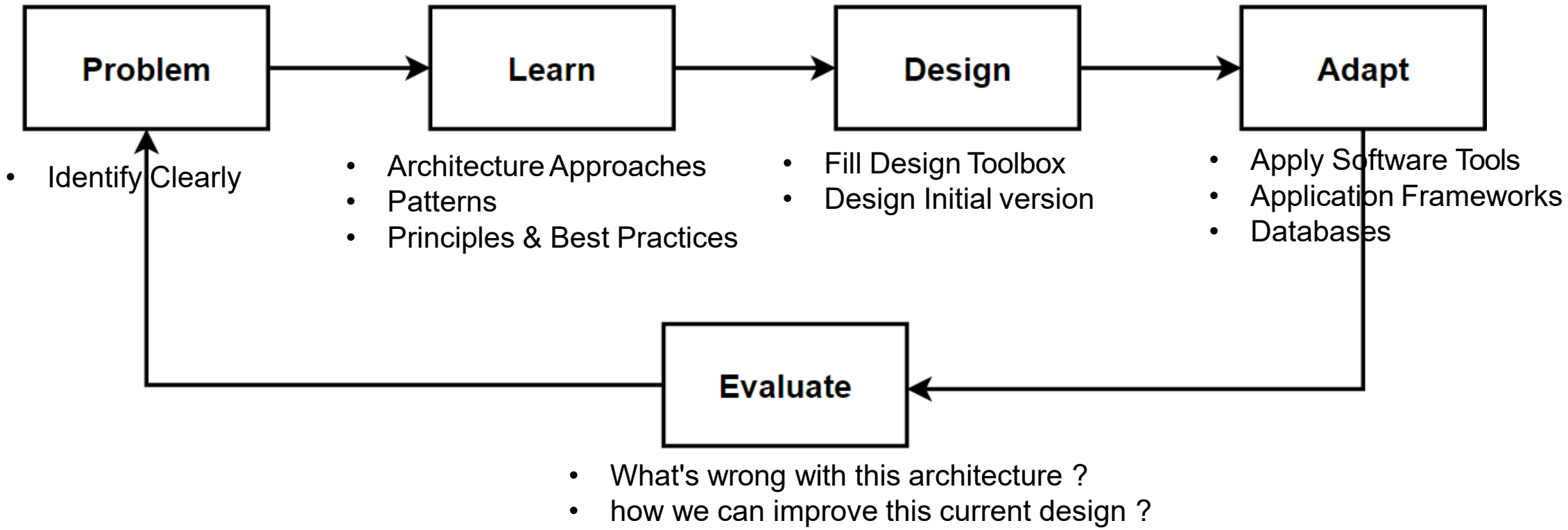
Backend Microservices
- Java – Spring Boot
- .Net – Asp.net
- JS – NodeJS



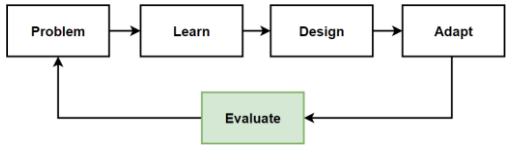
READ DB - No-SQL Database
- Cassandra – NoSQL Peer-to-Peer
- MongoDB – NoSQL Document DB
- Redis – NoSQL Key-Value Pair
- Amazon DynamoDB, Azure CosmosDB

WRITE DB - Relational Database
- PostgreSQL
- SQL Server
- Oracle

Way of Learning – The Course Flow

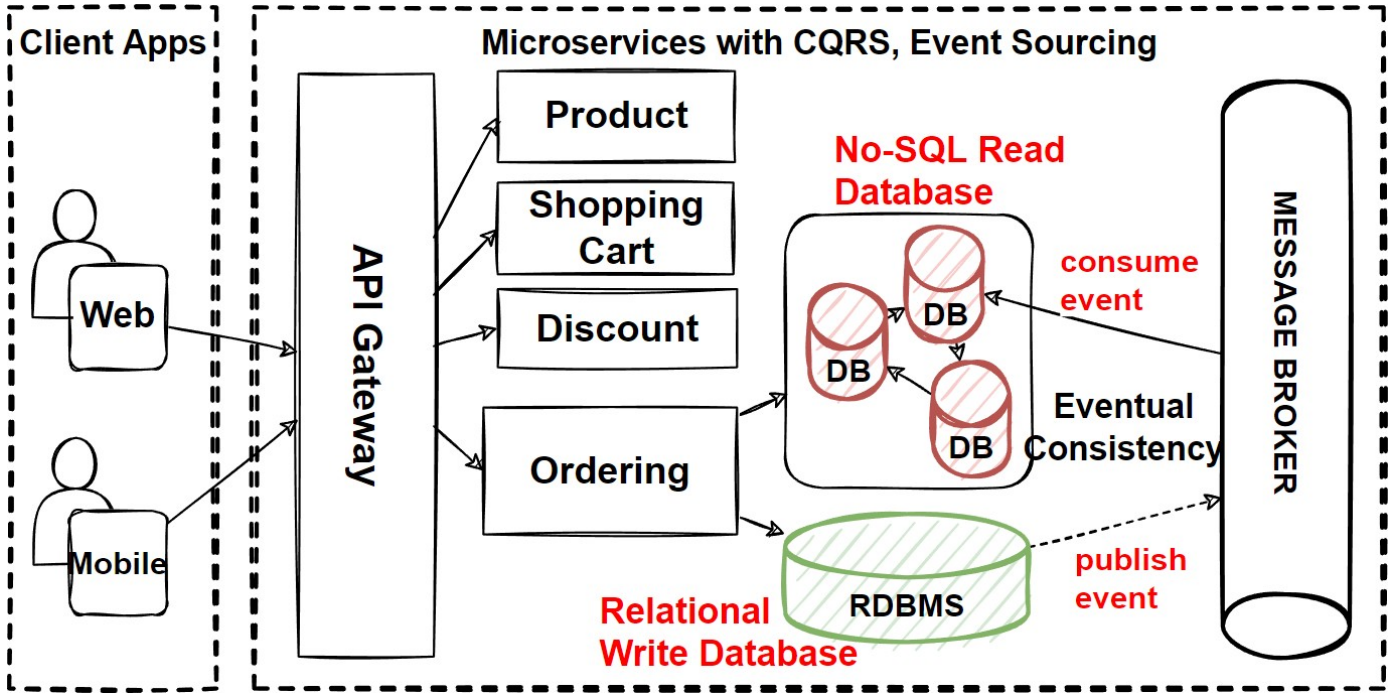


Evaluate: Microservice Architecture with CQRS, Event Sourcing, Eventual Consistency



Benefits

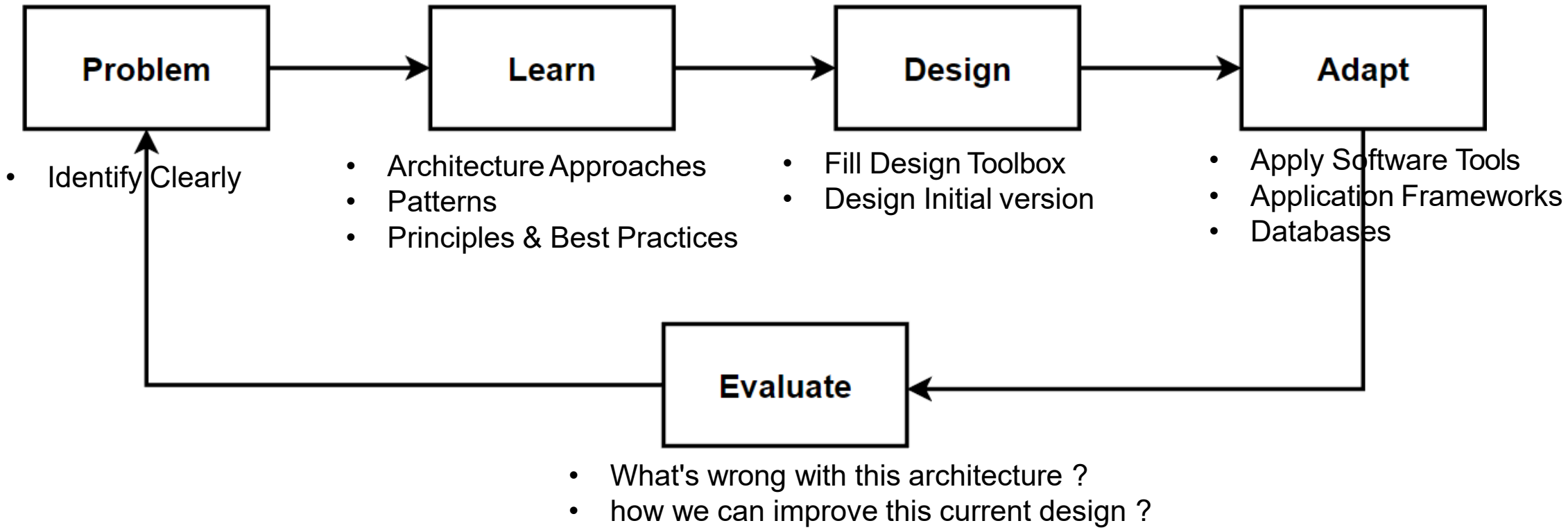
- Better Scalability, Separate Read and Write databases scales independently.
- Increased Query Performance, read database denormalized data reduce to complex and long-running join queries.
- Increased Maintability and Flexibility, system better evolve over time
- Distributed Horizontally Scaled Databases that ready for Kubernetes Deployments



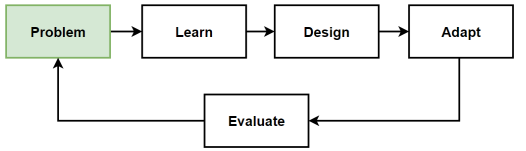
Drawbacks

- Increased Complexity, CQRS makes your system more complex design.
- Eventual Consistency
- Distributed Transaction Management

Way of Learning – The Course Flow

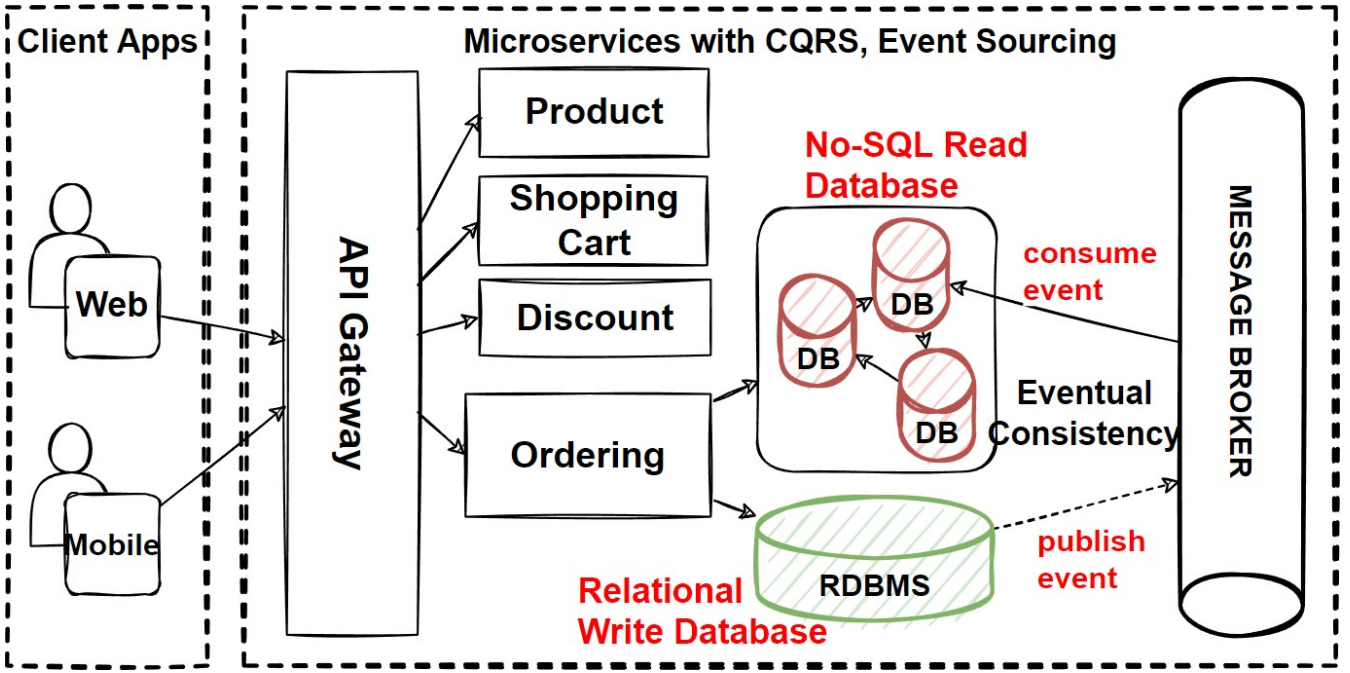


Problem: Manage Consistency Across Microservices in Distributed Transactions



Considerations

- Distributed Transactions that required to visit several microservices ?
- Consistency across multiple microservices ?
- Rollback transaction and run compensating steps ?



Problems

- Distributed Transaction Management
- Rollback Transaction on Distributed Environment
- Run Compensate Steps if one of service fail

Solutions

- Microservices Distributed Transaction Management Pattern and Best Practices
- Saga Pattern for Distributed Transactions
- Transactional Outbox Pattern
- Compensating Transaction pattern
- CDC - Change Data Capture

CÁM ƠN ĐÃ CHÚ Ý LẮNG NGHE!